

Chapter 0: Foundations and Prerequisites

Pre-LLM Multi-Agent Theory, Transformer Mechanics, and the Kernel Gap

The Authors

September 2026

Express Lane: Eight Theoretical Foundations. Before constructing an operating kernel for autonomous systems, we must establish the physical and theoretical ground upon which modern AI agents stand. This chapter establishes eight core foundations assumed by subsequent chapters:

1. **Dual Lineage of Agency:** Connecting classical symbolic multi-agent systems to generative connectionism (§1.3).
2. **Classical Multi-Agent Theory:** Bratman practical rationality, Rao–Georgeff BDI_{CTL}^* temporal logic, AgentSpeak(L) operational semantics, normative deontic reasoning, Hewitt–Agha actor concurrency versus Hoare CSP, FIPA-00037 speech acts, Smith’s Contract Net Protocol, Big Brother dynamic epistemic logic, and game-theoretic mechanism design (§2).
3. **The Physics of Modern LLM Execution:** Scaled dot-product attention mechanics, $O(L^2)$ quadratic complexity, KV-cache memory economics, training regimes (pretraining, SFT, RLHF/DPO), the Alignment Trap, sampling temperature entropy $(1 - (1 - \epsilon)^N \rightarrow 1)$, and externalized state in SQLite (§3).
4. **Modern Coding Agent Architectures, Planning, and Search:** Closed-loop POMDP deliberation (ReAct, Tree of Thoughts, Reflexion), lifelong skill acquisition (Voyager), topological DAG execution (TDAG), Hypertree Planning, PRMs, and multi-agent formal verification (§4).
5. **Tool Execution Hypervisors and Supervisory Hooks:** Model Context Protocol (MCP) substrate, progressive skill disclosure, and Ramadge–Wonham supervisory control hooks $(L(S/G) \subseteq K)$ (§5).
6. **Swarm Topologies, Coordination, and Multi-Agent Economics:** Stigmergic coordination, Conway’s Law in agent swarms, MetaGPT SOP roles, Six Sigma defect minimization, durable personas, and zombie salvage (§6).
7. **The Kernel Gap:** Mathematical proof of why generative autoregressive policies cannot act as their own operating kernel (§7).
8. **Chapter Map and Architectural Plates Portfolio:** Structural map of the volume and curated Swiss plates coda (§8).

1 Introduction: The Unit of Account, Agency, and the Kernel Gap

1.1 The Unit of Account is the Work

Agentic software is usually measured by the agent: capability, runtime, tool access, and parallel copies. That is the wrong unit of account. A process may disappear, crash, rename itself,

summarize away its mistakes, or silently hand work off to an untracked child process. The durable object of governance must be **the work**: its cryptographic authorization, permitted side effects, immutable evidence trail, open obligations, and ultimate liability for the physical outcome.

1.2 Defining the Agent and the Kernel: Mathematical Form and Architectural Necessity

To prevent category errors across this volume, we formalize both terms:

Definition 1.1 (Autonomous Agent). An *agent* α is a resource-bounded discrete-time policy $\pi_\theta(a_t \mid h_t)$ that consumes observation history $h_t \in \mathcal{H}$ and emits actions $a_t \in \mathcal{A}$ within a host environment. An agent maintains internal state (stochastically or symbolically) and interacts with physical resources strictly through mediating actuators.

Definition 1.2 (Operating Kernel). An *operating kernel* $\mathcal{K}$ is the minimal, privileged, crash-durable, and authoritative mediation substrate that:

1. Governs physical and logical resource allocation (CPU, memory, filesystem, network, locks, database handles);
2. Enforces execution invariants and capability boundaries that cannot be bypassed or overridden by tenant processes;
3. Serializes concurrent state mutations into an authoritative, verifiable, and crash-resilient causal history;
4. Preserves non-repudiable audit evidence of all operations crossing security boundaries.

1.3 The Dual Lineage of Autonomous Agency

Modern artificial intelligence did not begin with autoregressive transformers, nor did multi-agent coordination originate with chat completions. Autonomous software rests on two distinct, historically separated intellectual traditions:

1. **Classical Symbolic Multi-Agent Systems (1980–2005):** Grounded in mathematical logic, practical philosophy, and formal distributed systems. This lineage produced:
 - **Intentional Agency:** Bratman’s Belief-Desire-Intention (BDI) architecture (1987; see §2).
 - **Modal Action Logics:** Rao–Georgeff branching-time temporal logic (BDI_{CTL}^*) and AgentSpeak(L) operational semantics.
 - **Structured Concurrency:** Hewitt’s Actor Model (1973) and Hoare’s Communicating Sequential Processes (CSP, 1978).
 - **Protocol Engineering:** FIPA ACL speech act standards, Smith’s Contract Net Protocol (CNP, 1980), and Charrier et al.’s Big Brother Logic (2014).

The Failure Mode of Symbolic MAS: Classical systems had rigorous formal semantics, provable convergence, and ironclad safety guarantees, but were crippled by brittle real-world grounding. They could not parse arbitrary natural language, write production TypeScript, or diagnose unexpected compiler warnings.

2. **Generative Deep Learning (2017–Present):** Grounded in the transformer self-attention architecture (Vaswani et al., 2017; see §3), massive web-scale pre-training, and reinforcement learning alignment (RLHF/DPO). This lineage produced coding engines capable of synthesizing complex AST modifications, reading compiler diagnostics, and using terminal tools

in a closed loop (ReAct; see §4). *The Failure Mode of Generative Connectionism*: Modern coding agents possess remarkable semantic fluency, but exhibit zero innate commitment, catastrophic intention drift, probabilistic amnesia upon context window compaction, and non-deterministic adherence to security boundaries.

1.4 The Structural Deficit of Generative Models

Table 1 contrasts these two traditions. The core thesis of this volume is that **neither lineage alone can achieve safe autonomous engineering**. An LLM without an operating kernel is a chaotic, non-deterministic generator that will inevitably induce data corruption, race conditions, and security leaks. A symbolic system without an LLM cannot interact with the messiness of production codebases.

Architectural Dimension	Classical Symbolic MAS	Generative LLM Agents
Cognitive Substrate	Modal logic, first-order rules, unification	Dense multi-layer neural networks ($\sim 10^{11}$ params)
Goal Representation	Explicit syntactic desires and intention stacks	Latent prompt conditioning and reward steering
Commitment Dynamics	Formal axioms (blind, single-minded, open)	Soft stochastic retention; vulnerable to context drift
Concurrency Control	Message passing, rendezvous, Petri nets	Ad-hoc prompt warnings (“do not edit simultaneously”)
Failure Handling	Explicit recovery goals (e.g., $\neg!$ in AgentSpeak)	Probabilistic backtracking or hallucinated apologies
Security & Boundaries	Formal capability tokens, deontic permissions	System prompts easily bypassed via indirect injection
Physical Mutex	Database locks, OS semaphores	None; relies entirely on host process environment

Table 1: Comparative analysis of Classical Symbolic MAS versus Generative Neural Agents.

Therefore, functionally, mathematically, and architecturally, a persistent supervisory daemon that acts as the single-writer mediator of state, capability attenuation, and resource accounting **is an operating kernel at the application and coordination plane**.

1.5 Eight Propositions of Accountable Coordination

This book develops that claim in eight foundational propositions, one per chapter. Each proposition establishes a structural guarantee enforced by the operating kernel at the application and multi-agent coordination plane:

Proposition 1: Prevention Begins at the Effect Boundary (The Single-Writer Kernel)

Rules can be enforced before an act only when the system controls the unbypassable channel through which the act occurs. An operating system cannot rely on probabilistic language models to self-police or respect advisory prompt instructions such as “please do not edit simultaneously.” Without an authoritative mediator, concurrent uncoordinated agents inevitably induce the classic lost-update anomaly [18, 10]: Agent *A* reads version v_0 , Agent *B* reads v_0 , Agent *A* writes v_A , and Agent *B* subsequently overwrites with v_B , silently destroying Agent *A*’s work with zero compiler or runtime warning.

To guarantee mutual exclusion and crash resilience, all local mutations across shared repositories, lock registries, and port assignments must funnel through one serial write authority: an append-only write-ahead log (WAL) operating in SQLite. By serializing parallel mutation

requests into a single monotonically ordered causal sequence, the kernel guarantees strict serializability ($t_1 \prec t_2 \prec t_3$) and crash-stop consistency. Semantic failures that no gate can decide remain matters of detection and remedy, but physical race conditions are prevented structurally at the effect boundary.

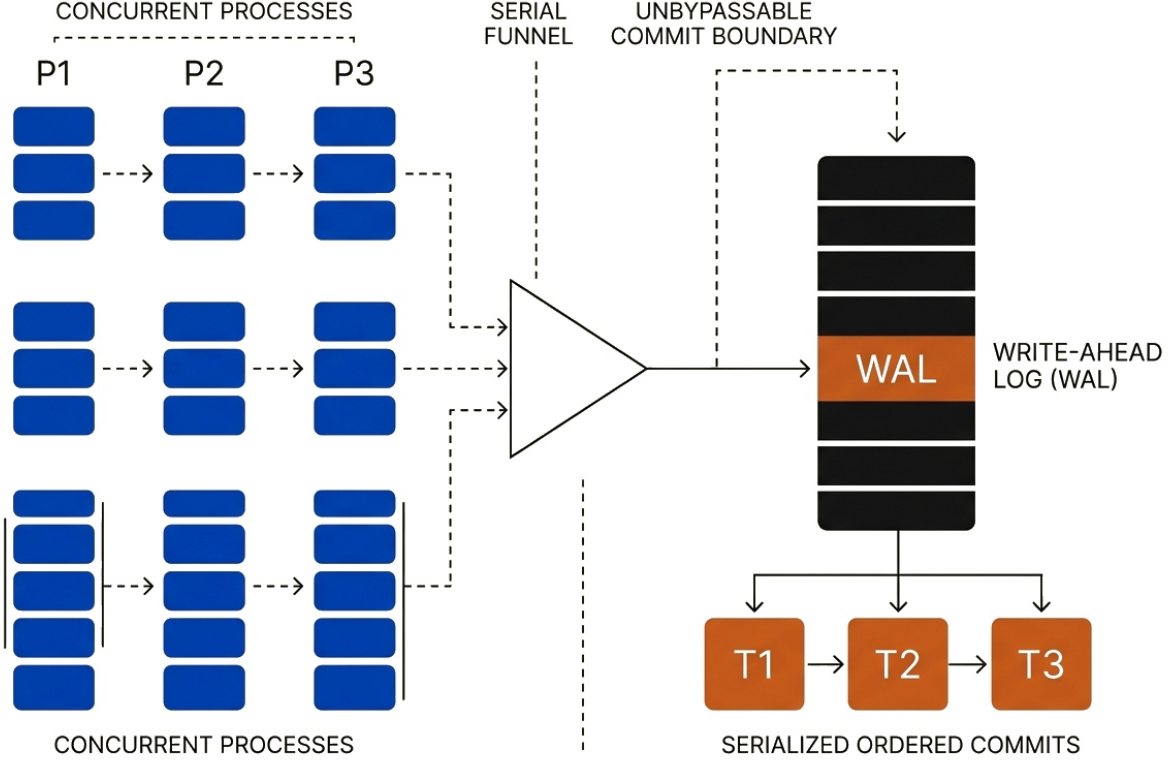


Figure 1: Proposition 1 Mechanics: Serial Write Arbitration. Parallel, uncoordinated writer attempts from independent agent processes (A_1, A_2, A_3) funnel into a single serialized write-ahead log (WAL) operated by the kernel daemon. By enforcing mutual exclusion at the physical effect boundary, race conditions and lost updates are eliminated before mutations reach persistent storage.

Formally, let $\tau = (e_1, e_2, \dots, e_n)$ be a concurrent execution trace of tool invocations generated by independent agent processes. Under Lamport’s happened-before relation $\prec$ [18], if two state-mutating events e_i, e_j access a shared resource R with $\neg(e_i \prec e_j) \wedge \neg(e_j \prec e_i)$, the execution contains an unmediated race. The Single-Writer Kernel introduces an out-of-band arbitration function $\mathcal{K}_{\text{WAL}} : \mathcal{E} \rightarrow \mathbb{N}$ that assigns a strict total order $\sigma(e) \in \mathbb{N}$ to every admitted mutation. Any write attempt that lacks a valid lock ticket is refused at the effect boundary with an explicit conflict receipt (409 **Conflict**), preventing silent corruption.

Proposition 2: Delegation Only Narrows (The Anchor Protocol)

In classical security engineering, authority must obey Saltzer and Schroeder’s fundamental Principle of Least Privilege [25]: every process must operate using the least set of privileges necessary to complete its assigned task. In naive multi-agent architectures where agents delegate subtasks by synthesizing unconstrained prompts, authority frequently inflates due to broad, ambiguous system instructions, leading to catastrophic privilege escalation.

Under the Anchor Protocol, every child capability is a strict, verifiable subset of its parent’s authority and validity lifetime ($C_0 \supset C_1 \supset C_2$ and $T_0 > T_1 > T_2$). Authentication proves who signed the credential; attenuation specifies what the signature is permitted to authorize. Chained cryptographic capability tokens (such as macaroons [7] and Harbor Cards) enforce monotonic

restriction at every delegation hop: an intermediate delegate cannot widen permissions, extend deadlines, or remove sandbox constraints when dispatching subordinate workers.

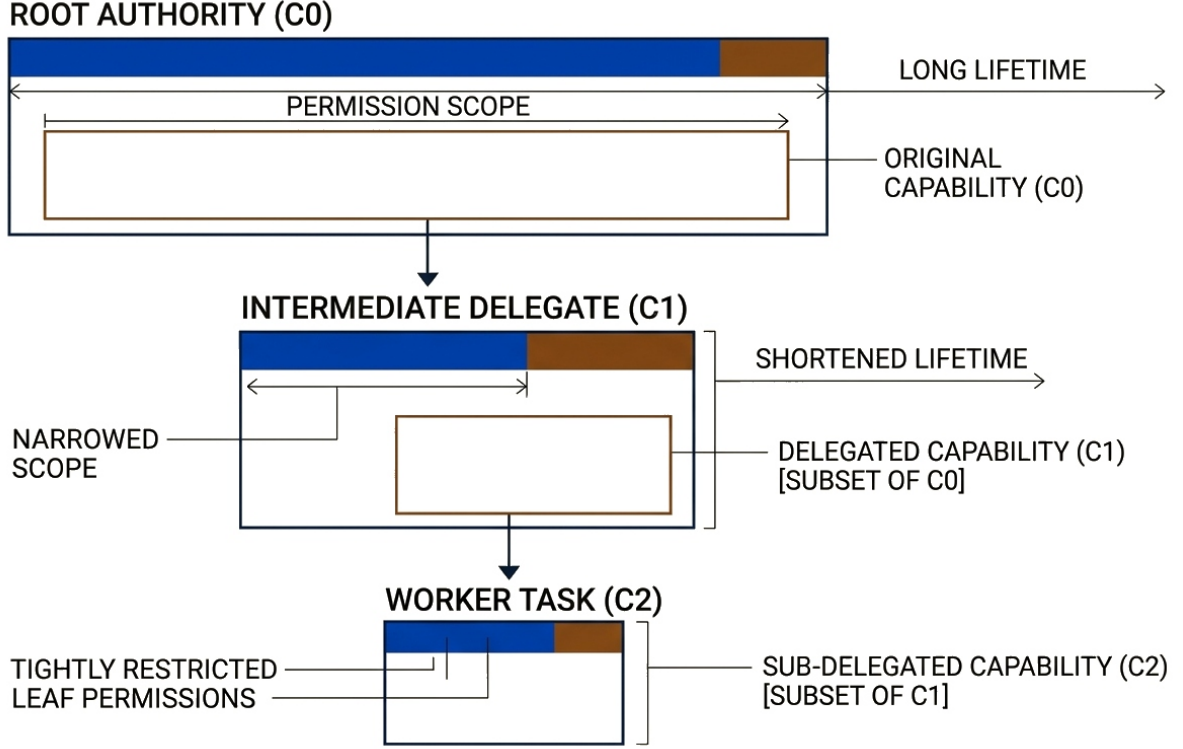


Figure 2: Proposition 2 Mechanics: Monotonic Capability Attenuation. Chained cryptographic capability tokens enforce monotonic restriction at every delegation hop. An intermediate delegate can append attenuating caveats (narrowing path scopes, reducing budgets, and shortening TTLs), but can never widen authority or extend parent lifetimes ($C_0 \supset C_1 \supset C_2$).

Mathematically, a capability token M_k at delegation depth k is constructed as an HMAC chain:

$$s_k = \text{HMAC}(s_{k-1}, c_k) \quad (1)$$

where s_0 is the root signing secret held exclusively by the supervisory kernel and c_k represents a contextual restriction (e.g., `path_prefix` $\subseteq$ `/src/lib`, `max_spend` $\leq$ `$0.50`, `expires` $\leq$ `t0 + 300s`). Because HMAC is a one-way cryptographic function, a delegate cannot strip existing caveats $c_1, \dots, c_k$ to restore wider authority without knowing the root key s_0 . Monotonic narrowing is thus cryptographically guaranteed across arbitrary delegation depths.

Proposition 3: Confinement Is a Property of the Room, Not of the Tenant (The Sealed Harbor)

In multi-tenant, multi-principal systems, mutual distrust is the necessary default. Two owners who trust neither each other nor the hosting venue can still take a verified computation out of a sealed room, provided the execution chamber guarantees non-bypassable physical isolation. Confinement cannot be promised by the tenant model: an LLM process is vulnerable to prompt injection, memory corruption, or adversarial jailbreaking.

Confinement must therefore be an invariant of the room itself, enforced by the operating system kernel (macOS Seatbelt, Linux Landlock and Bubblewrap). The sealed harbor severs all ambient network sockets and shared disk access, leaving a single, strictly metered egress slot: silence except through the slot, an unbypassable gate on the only enforceable boundary,

a conserved spend budget that fails closed, and synthetic canary tokens planted to detect and prove any covert side-channel data exfiltration. What the room cannot promise is stated as an explicit boundary, at the same prominence as the claims.

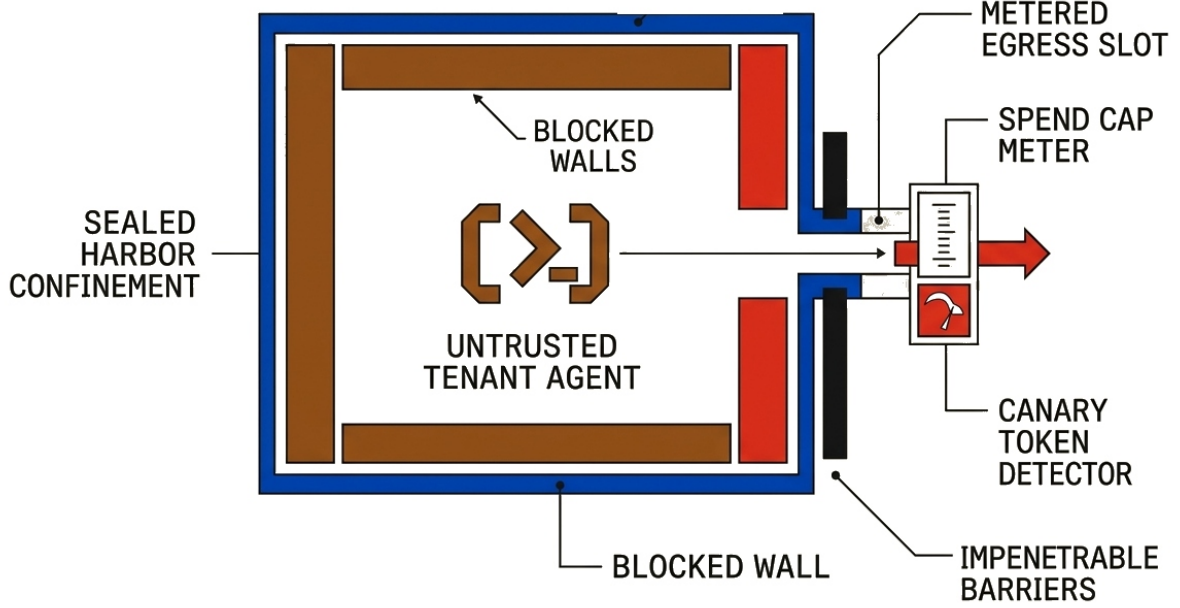


Figure 3: Proposition 3 Mechanics: Sealed Room Confinement. Operating system sandbox profiles (Seatbelt SBPL / Linux Landlock) enclose untrusted agent subprocesses, severing ambient network and shared disk paths. All outbound communication must traverse a single metered egress slot governed by canary token inspection and conserved spend caps.

The confinement policy is expressed as a non-bypassable sandbox profile P_{room} compiled into the kernel:

$$\forall \sigma \in \Sigma_{\text{syscall}}, \quad \text{Permit}(\sigma \mid P_{\text{room}}) \iff \sigma \in \Sigma_{\text{confined}} \quad (2)$$

where Σ_{confined} explicitly excludes socket creation, non-leased filesystem writes, and child process execution outside leased worktrees. Canary tokens $\kappa \sim \mathcal{U}(\{0, 1\}^{256})$ are embedded in local test environments; if a hash of κ is observed traversing the metered egress gateway, the tenant’s execution lease is instantly revoked and its security deposit slashed.

Proposition 4: Supervision Has an Information Cost (Attention and Supervision)

Human oversight is constrained by fundamental cognitive and information-theoretic limits. As Bainbridge observed in *Ironies of Automation* [5], automating routine operations does not eliminate human responsibility; it concentrates attention on rare, high-consequence failure modes. A human operator cannot parse a raw firehose of millions of agent telemetry tokens without suffering cognitive saturation and supervisory fatigue.

Yet naive LLM-generated summaries are actively dangerous: they hallucinate away subtle test failures, sanitize errors, and declare false victories. A digest cannot guarantee that a supervisor will notice any one of many relevant events unless it expends sufficient bits and attention to distinguish them. In an accountable architecture, a summary is therefore an index into evidence, not a substitute for it. The supervisory dashboard presents low-entropy summaries backed by cryptographic pointers directly into the underlying Merkle event log, allowing instant drilling down from high-level status to raw syscall traces.

Let $H(X)$ be the entropy of the agent’s raw execution trajectory and let D be a compressed supervisory digest. By the Data Processing Inequality, $I(X; Y) \leq H(D) \ll H(X)$, meaning

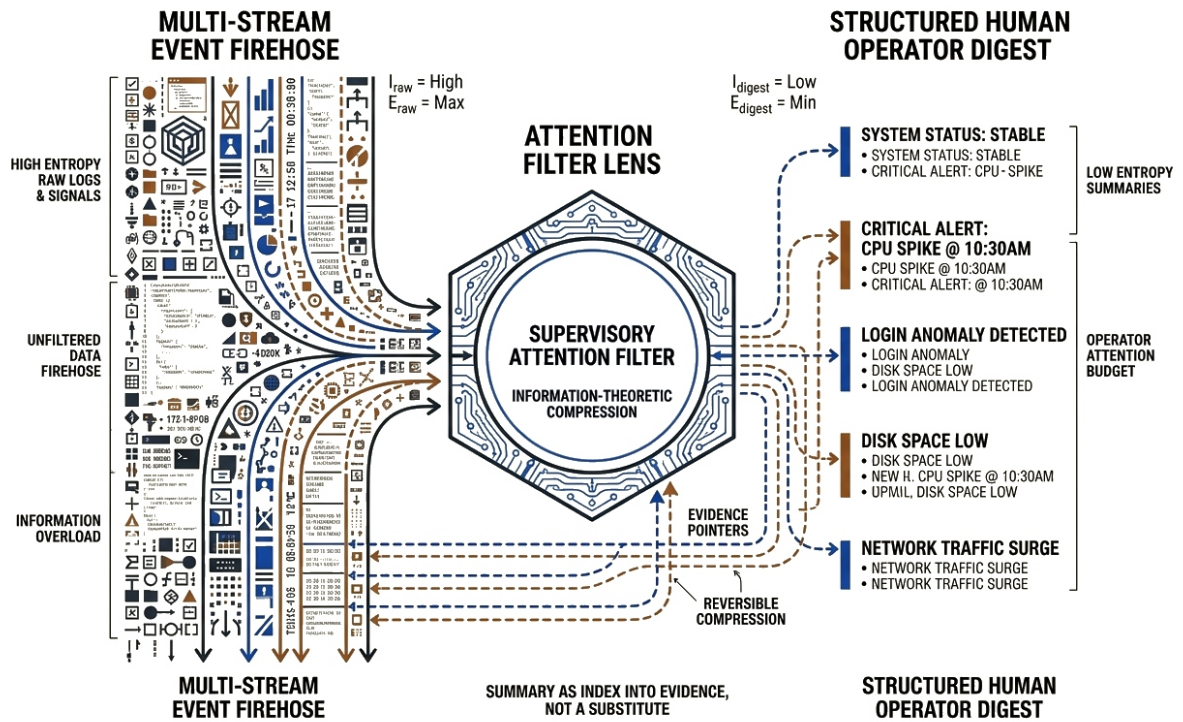


Figure 4: Proposition 4 Mechanics: Information Cost and Evidence Drilling. Supervisory digests operate as reversible, low-entropy projections indexed into an immutable Merkle evidence log. An operator can drill down with zero friction from high-level fleet status down to exact execution receipts and byte-level syscall traces.

information is unavoidably lost in any summary. To prevent supervisory blindness, the kernel constructs digests not as narrative text, but as a directed acyclic graph of cryptographic content digests $\mathcal{H}(e_i)$. The operator inspects a bounded view of complexity $O(\log N)$, with mathematical guarantees that any contested claim links directly to its underlying execution receipt ρ_i .

Proposition 5: Accountability Must Outlive the Process (From Spawn to Person)

Operating system processes are inherently mortal: they can be terminated by out-of-memory killers, abort on unhandled exceptions, or be restarted intentionally. If an agent’s identity, reputation, debt ledger, and liabilities exist only in the volatile RAM of a running Python or Node process, a defecting agent can escape consequences simply by crashing and rebooting under a new process ID.

An Agent Operating Kernel decouples ephemeral execution bodies from durable agent personas. A task acquires a durable cryptographic principal (Ed25519 keypair), an append-only capability history, a Merkle-backed evidence record, active normative obligations, and ultimate liability for physical outcomes. Restarting a crashed process or leasing a fresh container does not wipe the case file clean or replenish consumed token budgets: the durable persona resumes execution from the exact checkpoint recorded in the persistent ledger.

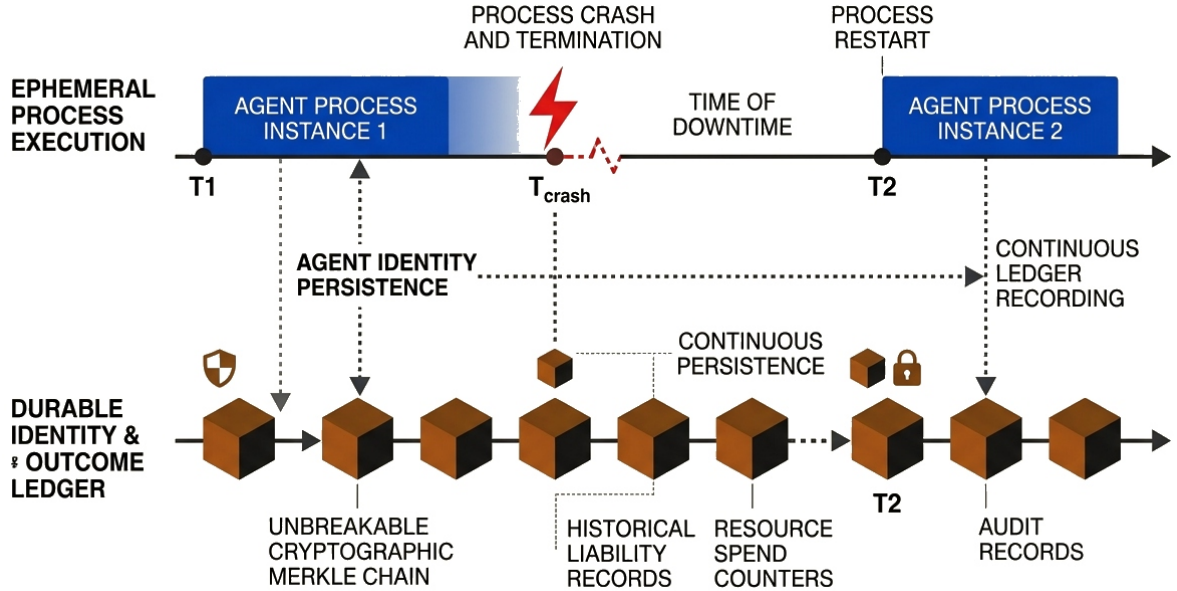


Figure 5: Proposition 5 Mechanics: Durable Persona vs. Ephemeral Process. Ephemeral container processes (PIDs) can crash, abort, or be recycled, but the durable agent persona (Ed25519 keypair, capability history, and debt ledger) persists in the append-only kernel database, resuming state seamlessly across failures.

Formally, an agent persona is defined as a persistent state tuple:

$$\mathcal{P} = \langle \text{pk}_{\text{agent}}, \mathcal{L}_{\text{history}}, \mathcal{O}_{\text{active}}, \mathcal{B}_{\text{consumed}}, \sigma_{\text{checkpoint}} \rangle \quad (3)$$

where pk_{agent} is the public signing identity, $\mathcal{L}_{\text{history}}$ is the append-only hash chain of past commitments, $\mathcal{O}_{\text{active}}$ is the set of unfulfilled obligations, and $\mathcal{B}_{\text{consumed}}$ is the non-resetting spend tally. When a container process terminates (SIGKILL), the kernel reclaims the ephemeral execution lease, but freezes $\mathcal{P}$. A successor process can only resume work by presenting the cryptographic proof of identity corresponding to pk_{agent} .

Proposition 6: Exchange Requires Separated Instruments (The Harbor Economy)

When autonomous agents trade compute, task assignments, and data access, economic coordination requires robust market design. In classical economics, Akerlof [2] demonstrated that asymmetric information and unseparated incentives degrade markets into a “market for lemons.” Conflating multiple financial functions into a single scalar reputation score or omnibus utility token creates acute moral hazard.

A trustworthy harbor economy strictly separates four distinct coordination instruments: (1) **Payment**, which compensates delivered labor; (2) **Collateral**, which bonds and prices bounded failure; (3) **Evidence**, which provides cryptographic proof of task completion; and (4) **Settlement**, which atomically and irrevocably closes obligations. Combining these functions into a single score undermines trust and invites strategic manipulation; separating them guarantees that claims can be audited independently of the currencies used to clear them.

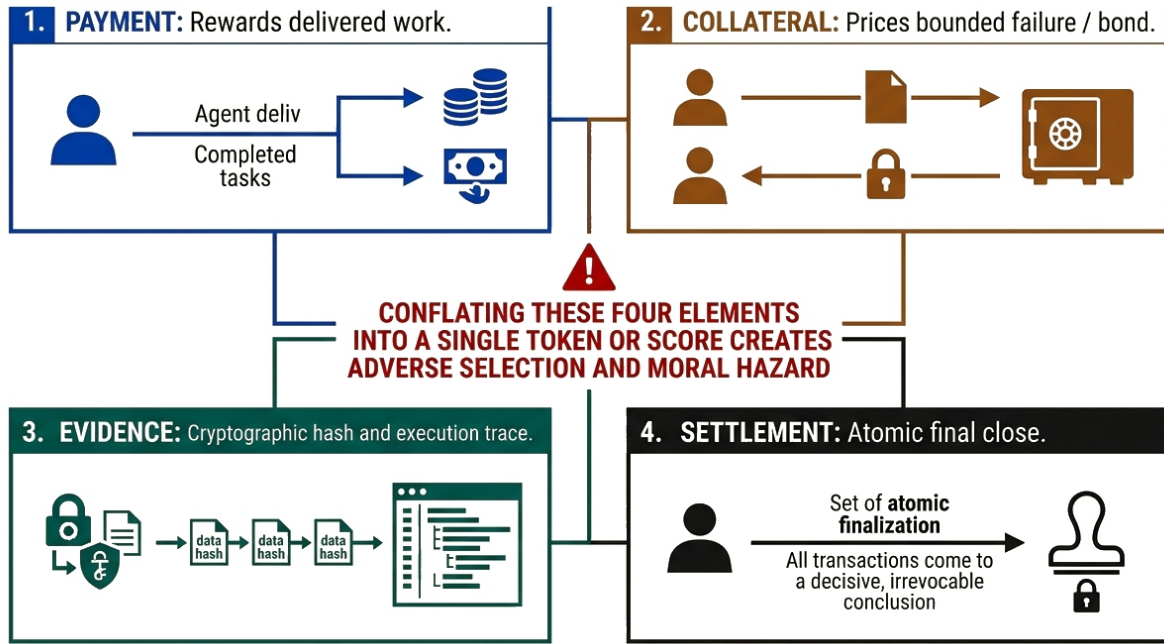


Figure 6: Proposition 6 Mechanics: Four Separated Market Instruments. To eliminate moral hazard and adverse selection in multi-agent trading, the kernel strictly separates Payment (labor compensation), Collateral (failure bonding), Evidence (execution proofs), and Settlement (atomic obligation closure).

Under this separated architecture, an agent offering a service must post collateral $C \geq \mathcal{V}(\text{risk})$ into escrow. When the task completes, the contractor produces evidence $\mathcal{E}$ (test logs, diff hashes). Only when $\mathcal{E}$ satisfies the verifier contract does settlement $\mathcal{S}$ execute, simultaneously disbursing payment P , returning collateral C , and updating the reputation ledger. Conflating payment and collateral into a single balance allows defaulting agents to consume resources without recourse; separating them enforces incentive compatibility.

Proposition 7: Uncertainty Needs an Institution (The Bonded Commons)

In complex software engineering, deterministic formal verification cannot decide every semantic property, and static type checkers cannot foresee every runtime interaction. Managing residual uncertainty requires institutional governance, grounded in Ostrom’s governing of shared commons [21] and Becker’s economics of deterrence [6].

Some harms cannot be prevented upfront, and some outcomes cannot be judged purely by mechanical test suites. To govern this irreducible uncertainty, the kernel constructs institutional procedures: bonded stake deposits, randomized spot audits by independent witness verifiers, neutral contestation and appeal arbitration, and strict conservation rules. By turning uncertainty into an inspectable, contested process backed by financial slashing, the commons deters defection while providing due process for false positives.

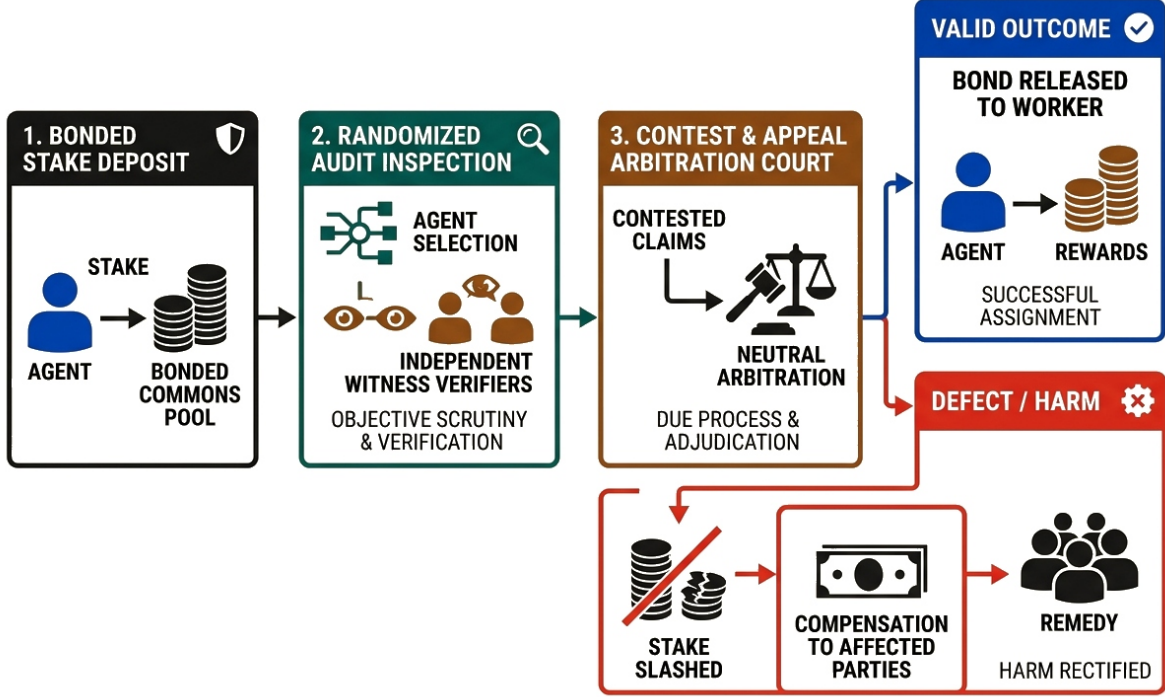


Figure 7: Proposition 7 Mechanics: Bonded Commons Governance. Shared host resources are governed as Common-Pool Resources (CPRs). Agents post bonded stake, undergo randomized independent verification audits, and face automated slashing upon invariant breach, balancing deterrence with due process.

Following Becker’s economic model of deterrence [6], the expected utility of defection $\mathbb{E}[U_{\text{defect}}]$ must be strictly negative:

$$\mathbb{E}[U_{\text{defect}}] = (1 - p_{\text{audit}}) \cdot G - p_{\text{audit}} \cdot S < 0 \quad (4)$$

where G is the gain from unauthorized shortcutting (e.g., skipping tests), p_{audit} is the probability of a randomized spot audit by a witness agent, and S is the financial stake slashed upon discovery. By tuning p_{audit} and S such that $S > \frac{1-p_{\text{audit}}}{p_{\text{audit}}} G$, the kernel sustains cooperation as a subgame-perfect Nash equilibrium.

Proposition 8: Federation Relays Evidence, Not Sovereignty (The Federated Harbor)

When autonomous fleets scale across distinct physical workstations, cloud tenancies, and organizational boundaries, centralizing all state into a single global database is both physically and politically impossible. Distributed machines operating under separate organizational ownership cannot submit to a single sovereign master.

Cross-machine federation must therefore exchange cryptographically verifiable evidence rather than shared state. Mutually distrustful hosts communicate across untrusted zero-trust relays,

exchanging signed capability attestations, revocation gossips, Merkle inclusion proofs, and settlement receipts without sharing a database. Transport carries non-repudiable evidence; the receiving host's local kernel retains complete sovereignty over whether to admit, execute, or reject the requested operation.

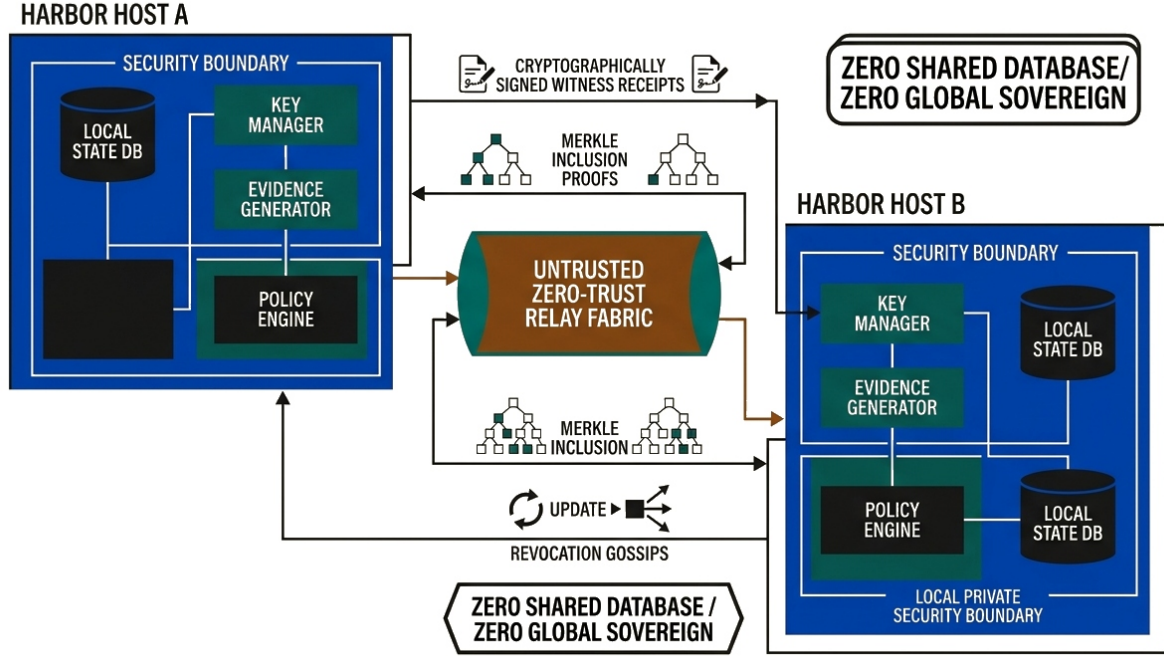


Figure 8: Proposition 8 Mechanics: Federated Evidence Relay. Cross-machine federation connects independent host kernels across zero-trust relays. Hosts exchange cryptographic proofs, witness receipts, and revocation gossips, retaining complete local sovereignty over resource admission without sharing a database.

In this federated topology, host kernels H_A, H_B maintain private, independent SQLite WAL databases. When Agent α on host H_A requests execution on host H_B , it transmits a signed credential package $\Pi = \langle \text{sig}_A(\alpha), \text{cap}_\alpha, \mathcal{M}_{\text{proof}} \rangle$. Host H_B 's local kernel independently verifies Π against its local trust anchor, checking revocation state without querying H_A 's internal database. If valid, H_B admits the task under local sandboxing rules. Sovereignty remains local; evidence crosses the wire.

Read these as a dependency chain: each chapter uses earlier mechanisms and states what they cannot guarantee.

1.6 What Counts as Knowing: Epistemic Bounds and Evidence Channels

Every consequential statement in this volume has a status and a boundary. Derivations establish relations under named assumptions. Finite checks exhaust a stated state space, not every deployment. Simulations estimate behavior under a stated generator; repository tests exercise named paths; running code proves availability, not universal mediation. Counterexamples remain in the record because they mark claims the research killed rather than conclusions it merely failed to prove.

Evidence is typed as well. When a chapter reports what a harbor observed, it names the channel that produced the observation, and each channel has a blind spot that no amount of volume removes:

- A *deterministic verifier* (a test, a type checker, a model checker) shows that a stated property

holds of the artifact it examined, and nothing about properties it did not state.

- A *provenance witness* (a signature, a hash, a Merkle inclusion, an append-only log) shows attribution and tamper-evidence relative to a commitment, never that every event was logged or that a description is true.
- A *protected meter* (a counter its subject cannot rewrite) shows a quantity, not its cause.
- *Expert judgment* shows an assessment under a named rubric and carries the assessor’s error rate.
- *Buyer preference* shows what was chosen, not that the choice was right.
- A *delayed outcome* shows what happened, and arrives too late to prevent it.

The reader should ask the same question of any dashboard: which channel, and what can it not show.

?? gives the implementation boundary; ?? records the results and their limits. The companion `docs/harbor-research/mega-volume-epistemic-manifest.yaml` preserves each result’s source, status, conditions, boundary, chapter home, and verification artifact. The chapters stand alone; the book’s claim is the chain they make.

2 Classical Multi-Agent Systems Foundations

Before analyzing neural coding agents, we formalize the foundational frameworks developed during the classical era of multi-agent systems.

2.1 Intentional Agency and Practical Rationality

Classical decision theory models an agent as an expected utility maximizer balancing subjective beliefs $\mathcal{B}$ and desires $\mathcal{D}$. In his seminal 1987 work *Intention, Plans, and Practical Reason*, philosopher Michael Bratman demonstrated that this two-component model is computationally inadequate for resource-bounded agents operating in dynamic environments [9].

A resource-bounded agent cannot continuously compute optimal policies from first principles at every millisecond. Practical rationality requires a third mental attitude: **Intention** ($\mathcal{I}$). Intentions are not merely desires with high utility; they represent *conduct-controlling pro-attitudes to which the agent has actively committed*.

Bratman established four defining functional roles of intentions:

1. **Drive Means-End Reasoning:** An intention to achieve goal ϕ obligates the agent to formulate sub-plans and allocate computational cycles until ϕ is satisfied.
2. **Filter of Admissibility:** Once an intention ϕ is adopted, candidate options inconsistent with ϕ (given current beliefs $\mathcal{B}$) are ruled inadmissible without evaluating their cost-benefit payoff.
3. **Stability and Commitment:** Intentions resist reconsideration in the absence of relevant environmental interrupts or belief updates, preventing computational paralysis.
4. **Coordination Scaffolding:** Intentions enable interpersonal coordination: other agents can safely form expectations regarding an agent’s future actions based on its declared commitments.

2.2 The Rao–Georgeff BDI Temporal Logic (BDI_{CTL^*})

Rao and Georgeff [23, 24] formalized Bratman’s theory into a multimodal branching-time temporal logic (BDI_{CTL^*}). A temporal world model $\mathcal{M}$ is defined as an 8-tuple:

$$\mathcal{M} = \langle W, T, \prec, U, \mathcal{B}, \mathcal{D}, \mathcal{I}, \Phi \rangle \quad (5)$$

where W is a set of possible worlds, T is a discrete set of time points ordered by tree-branching relation $\prec$, U is the universe of discourse, Φ interprets predicates across worlds and times, and $\mathcal{B}, \mathcal{D}, \mathcal{I} \subseteq W \times T \times W$ are accessibility relations for Beliefs, Desires, and Intentions.

For world $w \in W$ at time $t \in T$:

$$\mathcal{B}_t(w) = \{w' \in W \mid (w, t, w') \in \mathcal{B}\} \quad (6)$$

$$\mathcal{D}_t(w) = \{w' \in W \mid (w, t, w') \in \mathcal{D}\} \quad (7)$$

$$\mathcal{I}_t(w) = \{w' \in W \mid (w, t, w') \in \mathcal{I}\} \quad (8)$$

Semantics of the mental modalities at state $\langle w, t \rangle$:

$$\mathcal{M}, w, t \models \text{Bel}(\phi) \iff \forall w' \in \mathcal{B}_t(w), \mathcal{M}, w', t \models \phi \quad (9)$$

$$\mathcal{M}, w, t \models \text{Des}(\phi) \iff \forall w' \in \mathcal{D}_t(w), \mathcal{M}, w', t \models \phi \quad (10)$$

$$\mathcal{M}, w, t \models \text{Int}(\phi) \iff \forall w' \in \mathcal{I}_t(w), \mathcal{M}, w', t \models \phi \quad (11)$$

Beliefs satisfy the *KD45* axiom system (deductive closure **K**, consistency **D**, positive introspection **4**, and negative introspection **5**). Desires and Intentions satisfy seriality (**D**), ensuring an agent never desires or intends logical contradictions ($\neg \text{Des}(\perp)$ and $\neg \text{Int}(\perp)$).

2.2.1 Compatibility and Weak Realism

In rational agency, an agent’s desires and intentions must be constrained by its beliefs. Under the **Weak Realism** axiom, an agent cannot intend what it believes to be impossible:

$$\forall w \in W, \mathcal{I}_t(w) \cap \mathcal{B}_t(w) \neq \emptyset \quad (12)$$

Syntactically, for temporal achievement goals:

$$\text{Int}(\mathbf{A}\Diamond\phi) \rightarrow \neg \text{Bel}(\neg \mathbf{E}\Diamond\phi) \quad (13)$$

where $\mathbf{A}\Diamond\phi$ denotes “ ϕ holds eventually on all future paths,” and $\mathbf{E}\Diamond\phi$ denotes “ ϕ holds on at least one branch.”

2.3 Formal Commitment Strategies

A commitment strategy dictates the precise formal conditions under which an agent drops an intention ϕ . Rao and Georgeff [23] formalized three canonical strategies:

Commitment Strategy	Formal Drop Axiom
Blind (Fanatical)	Maintains $\text{Int}(\mathbf{A}\Diamond\phi)$ until $\text{Bel}(\phi)$ holds. $\text{Int}(\mathbf{A}\Diamond\phi) \wedge \neg \text{Bel}(\phi) \rightarrow \mathbf{A}(\text{Int}(\mathbf{A}\Diamond\phi) \mathcal{U} \text{Bel}(\phi))$. <i>Pathology:</i> Induces infinite loops if ϕ becomes physically impossible.
Single-Minded	Drops ϕ if achieved <i>or</i> believed impossible. $\text{Int}(\mathbf{A}\Diamond\phi) \wedge \neg \text{Bel}(\phi) \wedge \text{Bel}(\mathbf{E}\Diamond\phi) \rightarrow \mathbf{A}(\text{Int}(\mathbf{A}\Diamond\phi) \mathcal{U} (\text{Bel}(\phi) \vee \neg \text{Bel}(\mathbf{E}\Diamond\phi)))$. <i>Standard:</i> Balances persistence with dynamic feasibility.
Open-Minded	Drops ϕ if achieved, impossible, <i>or no longer desired</i> . $\text{Int}(\mathbf{A}\Diamond\phi) \wedge \dots \wedge \text{Des}(\mathbf{A}\Diamond\phi) \rightarrow \mathbf{A}(\text{Int}(\mathbf{A}\Diamond\phi) \mathcal{U} (\text{Bel}(\phi) \vee \neg \text{Bel}(\mathbf{E}\Diamond\phi) \vee \neg \text{Des}(\mathbf{A}\Diamond\phi)))$.

Table 2: Canonical commitment strategies in BDI agent architectures.

🔒 Design Invariant 2.1 (Commitment Persistence).

An agent's intention $\iota \in \mathcal{I}$ must not be abandoned merely because a single step incurs a transient tool failure. Intention reconsideration is an explicit state transition triggered by invariant failure or objective discharge, not an accidental consequence of context window truncation.

2.4 Operational Semantics of AgentSpeak(L) and Normative Extensions

AgentSpeak(L) [8] provides structural operational semantics (SOS) for executing BDI agents. An agent configuration is a tuple $C = \langle bs, ps, is, \mathcal{E}, A \rangle$, where bs is the belief base, ps is the plan library, is is the set of active intention stacks, $\mathcal{E}$ is the event queue, and A is the action history.

The operational execution cycle is governed by three deterministic or priority-driven selection functions:

1. $S_E(\mathcal{E}) = \langle te, i \rangle$: Selects an event from $\mathcal{E}$ (e.g., belief change $+b$, goal addition $+\!g$).
2. $S_O(\text{App})$: Selects a plan (p, θ) from the applicable plans whose context is entailed by $bs\theta$.
3. $S_I(is) = i$: Selects which intention stack advances its execution step in the current cycle.

When an action a in plan p fails during host execution, AgentSpeak semantics do not silently abort the agent; instead, the failure transition posts a failure event $-\!g$ to $\mathcal{E}$, popping the failed plan frame and invoking explicit recovery plans:

$$\frac{\epsilon = \langle te, i \rangle, \quad \text{App}(\text{Rel}(te, ps), bs) = \emptyset}{\langle bs, ps, is, \mathcal{E}, A \rangle \longrightarrow \langle bs, ps, is, \mathcal{E} \cup \{ \langle -\!g, \text{pop}(i) \rangle \}, A \rangle} \quad (14)$$

Figure 9 synthesizes this operational choice architecture, illustrating how autonomous systems select between reactive rules, static classical planning, and dynamic BDI execution.

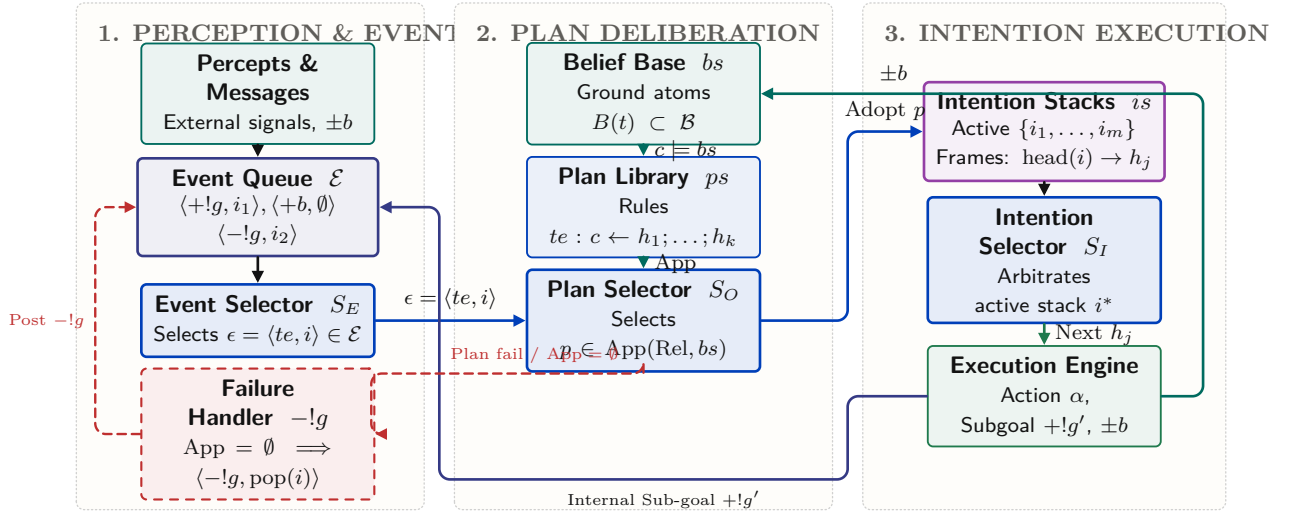


Figure 9: BDI Execution Flow and Operational Choice Architecture. Grounded in real-world software engineering workflows, adapted from AgentSpeak(L) semantics (Bordini et al., 2007). Contrasting reactive rule triggers, static STRIPS-style planners, and dynamic BDI agents. The three core selection functions (S_E for events, S_O for applicable plans, and S_I for intention stacks) arbitrate runtime goal scheduling, with explicit plan failure handling via the $-\!g$ recovery event.

2.4.1 Normative Extensions: Deontic Constraints and Regret Minimization

In institutional multi-agent systems, agents are constrained by external norms (deontic logic). A norm is defined as a 6-tuple $N = \langle \text{id}, \text{Type}, \phi_{\text{act}}, \phi_{\text{tgt}}, d, \text{Sanction} \rangle$, where $\text{Type} \in \{\mathcal{O}, \mathcal{F}, \mathcal{P}\}$ (Obligation, Prohibition, Permission), ϕ_{act} is the activation context, ϕ_{tgt} is the target state, d is the deadline, and Sanction is the penalty for violation.

To reconcile institutional norms with internal intentions, agents maintain an **Abstract Norm Base** (ANB) and evaluate instantiated norms in the **Norm Instance Base** (NIB). When norm instances conflict with active goals ($\text{Plans}(is \cup N) = \emptyset$), the agent resolves the normative conflict by partitioning candidate norms into **Maximal Non-Conflicting Subsets** (MNCS), selecting the subset that minimizes maximum penalty under a **minimax regret** criterion:

$$S^* = \arg \min_{S \in \Omega} \max_{n_v \in N \setminus S} \text{Cost}(\text{Sanction}(n_v)) \quad (15)$$

This ensures that any deliberate norm violation is recorded in an immutable ledger alongside its formal justification.

2.4.2 Why BDI Matters for LLMs: Curing Intention Drift and Hallucinatory Abandonment

Why should modern software engineers, operating cutting-edge transformer models, care about a symbolic multi-agent theory formulated in 1987? The answer lies in the acute operational pathologies of unanchored autoregressive inference.

Stateless large language models have no innate concept of commitment. Conditioned strictly on next-token likelihood, an unanchored model can formulate an elegant multi-step refactoring plan at step t , only to completely abandon it at step $t + 1$ upon encountering a minor compiler warning or linter diagnostic. In empirical developer workflows, models frequently exhibit three fatal failure modes:

1. **Intention Drift:** As intermediate tool output scrolls across the prompt buffer, attention dilution causes the agent to lose focus on its primary objective, drifting into endless exploratory side-quests.
2. **Spurious Abandonment:** Encountering an unexpected error code, an unanchored agent often prematurely declares a task “impossible” and hallucinations a false victory, or completely resets its approach, discarding hundreds of lines of valid working diffs.
3. **Infinite Re-planning Loops:** Lacking explicit commitment axioms, models repeatedly re-generate the same high-level task plan rather than executing intermediate action steps.

BDI practical rationality [9, 23] provides the exact mathematical scaffold required to stabilize neural agents:

- **What We Adopt (Single-Minded Commitment):** In an Agent Operating Kernel, the user’s high-level goal is converted into a durable intention $\iota \in \mathcal{I}$ registered in the kernel’s state machine. The agent operates under **single-minded commitment**: it is forbidden from abandoning ι unless it produces proof that ι has been satisfied (e.g., passing test suite receipts) or an unrecoverable invariant failure proves $\neg \mathbf{E} \Diamond \iota$ (e.g., compile error in upstream vendor package).
- **What We Reject (Rigid Symbolic Plan Libraries):** Classical BDI required human programmers to hand-code every applicable plan $p \in ps$ in advance. We reject this rigidity. In our hybrid architecture, the neural language model acts as an open-ended, highly creative *plan generator* and *replanning engine*, while the deterministic BDI kernel enforces the *intention stack*, monitors commitment persistence, and drives the $\neg !g$ failure recovery loop.

2.5 Concurrency Foundations: Hewitt–Agha Actors vs. Hoare’s Communicating Sequential Processes

When multiple agents deliberate and execute concurrently on a developer’s workstation, the concurrency substrate dictates whether the swarm operates harmoniously or degenerates into deadlocks and corrupted file state. Distributed systems research provides two foundational, mathematically rigorous concurrency paradigms:

- **Hewitt–Agha Actor Model (1973, 1986) [16, 1]:** The Actor Model treats the **actor** as the universal primitive of concurrent computation. An actor is an autonomous, computational entity that encapsulates private state, a mailbox (message queue), and behavior. In response to a message, an actor can concurrently perform exactly three operations:
 1. *Send* a finite number of messages to other actors with known addresses;
 2. *Create* a finite number of new actors;
 3. *Designate the replacement behavior* (**become**) to be used for the next message it receives.

Actors share **no mutable state**. Message passing is fundamentally **asynchronous**: sending is non-blocking, and messages are buffered in the recipient’s mailbox.

- **Hoare’s Communicating Sequential Processes (CSP) (1978) [17]:** CSP formalizes concurrent computation around **processes** communicating over synchronous, unbuffered **channels**. The core principles of CSP are:
 1. *Synchronous Rendezvous*: Communication is an unbuffered handshake. The output event $c!v$ on channel c and the input event $c?x$ must occur simultaneously ($\Delta t = 0$). Whichever process reaches the communication point first blocks until the other process arrives.
 2. *Guarded Choice (Alternative)*: A process can wait on multiple communication events simultaneously using the alternative construct (**alt**), proceeding with whichever channel becomes ready first.

Figure 10 details the dual concurrency architecture: Hewitt–Agha virtual actor isolation for stochastic deliberation, and Hoare’s Communicating Sequential Processes (CSP) synchronous rendezvous for physical state mutations.

2.6 From Asynchronous Isolation to Synchronous Rendezvous

The fundamental divergence between Hewitt’s Actor model and Hoare’s Communicating Sequential Processes reflects an architectural trade-off between *isolation* and *synchronization*:

1. **Asynchronous Mailboxes vs. Unbuffered Barriers:** In the Actor model, communication is non-blocking and decoupled in time. An agent posts a message to a peer’s mailbox and immediately resumes local execution. This temporal decoupling is indispensable for long-running neural inference loops: if an orchestrator were forced to block synchronously while a worker generated hundreds of tokens across a remote API, the entire supervisory loop would freeze. However, unbounded asynchronous mailboxes introduce severe hazards: message ordering is non-deterministic, queue backpressure is difficult to meter, and an agent can be flooded with stale context deltas.
2. **Synchronous Rendezvous for Scarce Resources:** Conversely, Hoare’s CSP enforces synchronous rendezvous: a send operation on an unbuffered channel cannot complete until a corresponding receive operation simultaneously executes. While synchronous rendezvous is prohibitive for stochastic agent inference, it is the mathematically optimal primitive for arbitrating scarce, physical, and non-fungible host resources: acquiring exclusive TCP port

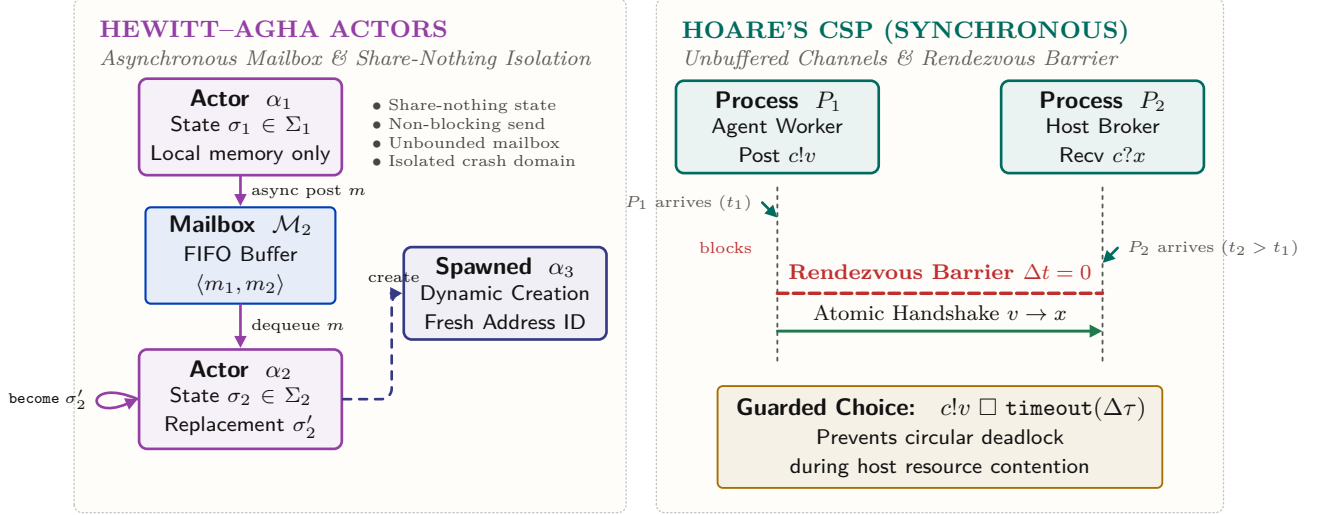


Figure 10: Concurrency Foundations for Multi-Agent Operating Kernels. Synthesizing Hewitt–Agha virtual actor isolation (asynchronous mailboxes, capability tokens, and `become` state transitions) with Hoare’s Communicating Sequential Processes (synchronous rendezvous barriers, $\Delta t = 0$, and guarded external choice with lease timeouts). Sovereign actors deliberate in isolated sandboxes, while physical host resource contention (TCP ports, SQLite WAL writes, git locks) is arbitrated via synchronous CSP rendezvous.

leases, linearizing commits in the SQLite write-ahead log (WAL), and locking git branch heads.

3. **The Synthesis in an Agent Operating Kernel:** Modern agent operating kernels synthesize both lineages. As illustrated in Figure 10, workers execute as sovereign actors within seatbelted processes, receiving tasks through capability-attenuated mailboxes and recording state transitions via `become`. Yet, whenever an actor attempts to alter shared reality, its request is arbitrated through Hoare-style synchronous barriers, guaranteeing mutual exclusion, preventing circular wait deadlocks through explicit lease timeouts, and ensuring deterministic linearizability.

2.6.1 Why Concurrency Lineages Matter: The Dual-Plane Operating Synthesis

Why should an autonomous agent infrastructure combine two historically competing concurrency models rather than adopting one exclusively? The answer is dictated by the physical constraints of developer machines and neural inference:

- **Why Pure CSP Fails for AI Agent Fleets:** If we built our multi-agent system purely on Hoare’s Communicating Sequential Processes (as in Go channels or Occam), communication requires synchronous rendezvous ($\Delta t = 0$). Autoregressive token generation across frontier APIs is highly stochastic, with latencies ranging from 500 milliseconds to tens of seconds. Under pure CSP, an orchestrator process that sends a message to a worker is paralyzed until the worker consumes it. If two agents simultaneously attempt to query each other, or if a child process hangs on a remote API timeout, the entire swarm freezes in an irrecoverable circular rendezvous deadlock.
- **Why Pure Actors Fail on Local Workstations:** Conversely, if we built our system purely on Hewitt–Agha actors (as in Erlang/OTP or Akka), all communication is asynchronous and all state is private. But a developer workstation is not an abstract cluster: it possesses scarce, non-fungible, shared physical resources. There is only one TCP port 3000, one SQLite database file on disk, and one `.git/index` lock file. If five autonomous

actors concurrently execute bash tool calls attempting to bind port 3000 or mutate the git repository without a synchronous arbitration barrier, the result is catastrophic: silent port squatting, broken working directories, and corrupted transaction logs.

- **What We Adopt in Port Daddy:** We implement a strict **Dual-Plane Architecture**. In the Deliberation Plane, agents are Hewitt–Agha actors: each subagent runs in an isolated OS process with private memory, communicating asynchronously over buffered mailbox queues and updating its internal state machine via `become`. In the Physical Actuation Plane, all interactions with scarce host resources pass through Hoare CSP rendezvous barriers enforced by the supervisory daemon: atomic mutexes, lease timeouts, and single-writer commit gates that guarantee mutual exclusion and linearizability.

2.7 Communicative Action and Speech Acts (FIPA-00037)

Standard computer networks exchange raw byte packets; autonomous multi-agent engineering systems exchange **speech acts**. Grounded in Austin’s distinction between locutionary (syntactic utterance), illocutionary (communicative intent), and perlocutionary (consequential effect) acts [4], and Searle’s taxonomy of illocutionary forces [27], the Foundation for Intelligent Physical Agents standardized the **FIPA ACL Communicative Act Library** (FIPA-00037) [13].

In a developer’s autonomous coding environment, speech acts are not abstract philosophy—they are the formal wire protocol between orchestrator agents, specialized coding subagents, and supervisory daemons. Every communicative act $a(s, r, \phi)$ between sender s and receiver r is formally defined by its **Feasibility Preconditions** (FP) and intended **Rational Effect** (RE). In an operating kernel, performatives cease to be informal chat tokens; they are cryptographically signed protocol events backed by content-addressed receipts. Consider a concrete developer scenario: an Orchestrator agent coordinates with a Database Supervisory agent to execute a non-blocking schema migration ("add_mfa_col") while handling concurrent transaction load and generating synchronized TypeScript types. Table 3 articulates these core performatives alongside their epistemic semantics:

Act	Category	Feasibility Precondition (FP)	Rational Effect (RE)
inform	Assertive	$Bel_s(\phi) \wedge \neg Bel_s(Bel_r(\phi))$	$Bel_r(\phi)$
request	Directive	$FP(\alpha)[s \setminus r] \wedge Bel_s(Agent(r, \alpha))$	$Done(\alpha)$
query-if	Directive	$\neg Bel_s(\phi) \wedge \neg Bel_s(\neg \phi)$	$Bel_s(Bel_r(\phi) \vee Bel_r(\neg \phi))$
propose	Commissive	$Bel_s(Int_s(Done(\alpha, \phi)))$	$Bel_r(Int_s(Done(\alpha, \phi)))$
agree	Commissive	$Bel_s(Int_s(Done(\alpha)))$	$Bel_r(Int_s(Done(\alpha)))$
refuse	Directive	$Bel_s(\neg Int_s(Done(\alpha)))$	$Bel_r(\neg Int_s(Done(\alpha)))$
failure	Assertive	$Bel_s(Failed(\alpha, \psi))$	$Bel_r(Failed(\alpha, \psi))$

Table 3: FIPA-00037 communicative performatives with mental-state preconditions and rational effects.

2.7.1 Why Speech Acts Matter: From Conversational Fluff to Cryptographic Envelopes

In naive multi-agent frameworks, agents communicate by exchanging raw natural language strings: “*Hey @database-agent, can you please run the migration and let me know when it’s done?*” This approach represents an acute engineering hazard:

1. **Syntactic and Pragmatic Ambiguity:** Natural language messages have fuzzy illocutionary force. Is the message an imperative command, a tentative proposal, or an informational suggestion? When an agent replies “*I’ll look into it,*” has it committed to an intention or merely acknowledged receipt?

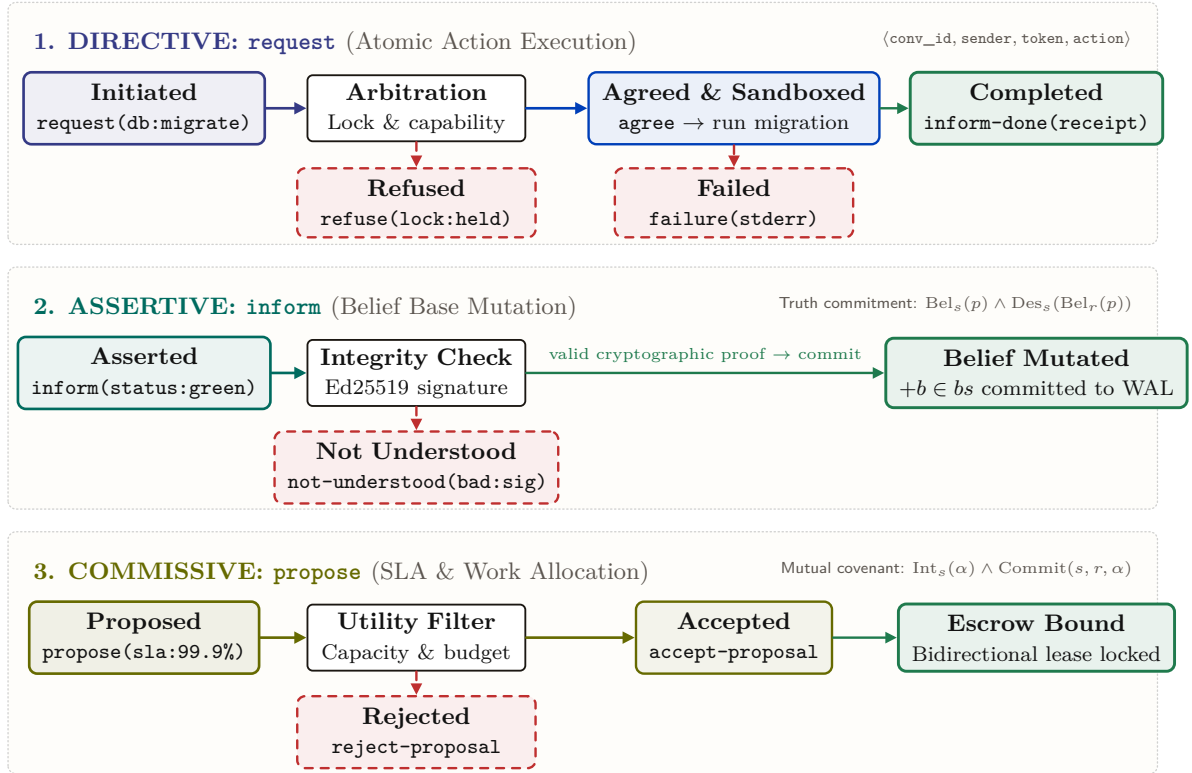


Figure 11: FIPA-00037 Communicative Speech Act State Transition Machine. Grounded in practical software engineering payloads. Top: Directive **request** lifecycle executing an atomic database migration under lock arbitration. Middle: Assertive **inform** directly mutating an agent’s belief base. Bottom: Commissive **propose** negotiating an SLA for canary deployment routing. Every envelope carries cryptographic sender/receiver verification, monotonic capability tokens, and explicit conversation correlation IDs.

2. **Conversational Fluff and Token Saturation:** Politeness tokens, preamble acknowledgments, and conversational pleasantries rapidly fill scarce prompt context windows, triggering premature compaction and attention dilution.
3. **Vulnerability to Conversational Injection:** When communication consists of free-form text, an untrusted worker agent can craft adversarial dialogue prompts that manipulate the recipient’s deliberation policy.

The FIPA-00037 standard [13] provides the exact structural solution:

- **What We Adopt (Explicit Performative Envelopes):** We adopt FIPA’s foundational illocutionary taxonomy: every message has a deterministic performative verb (**request**, **inform**, **propose**, **agree**, **refuse**, **failure**). In Port Daddy’s Swarm Radio and Tube messaging fabrics, messages are never raw strings; they are structured binary or JSON envelopes containing the sender’s cryptographic public key, recipient identifier, conversation correlation ID, performative type, and a typed payload conforming to a strict schema. An agent receiving a **request** knows with mathematical certainty that it is being asked to execute an action; an agent receiving an **agree** knows that a binding commitment has been formed.
- **What We Reject (Heavyweight Semantic Languages and Ontologies):** Classical FIPA required complex modal-logic semantic languages (FIPA SL) and heavyweight shared ontologies that required theorem provers to decode. We reject this runtime overhead. In our kernel, message bodies are strongly typed using Protocol Buffers or JSON Schemas, validated at wire-speed by the daemon before delivery, and cryptographically witnessed via Ed25519 signatures.

2.8 Task Allocation: Smith’s Contract Net Protocol (1980)

Reid G. Smith’s (1980) **Contract Net Protocol (CNP)** [30] formalized bilateral market-inspired negotiation between a Manager and distributed Contractor agents. In autonomous software fleets, CNP is the canonical mechanism for allocating urgent developer tasks—such as triage of a security CVE or compilation of an isolated package—across competing agent workers:

1. **Call for Proposals (CFP Broadcast):** The Manager broadcasts an announcement $\langle \text{task_id}, \text{eligibility}, \text{spec}, t_{\text{expire}} \rangle$ specifying required toolsets, maximum spend budget, and deadline.
2. **Bidding and Capability Self-Assessment:** Eligible contractor agents inspect their current local capacity, sandboxed tools, and model context before submitting formal bids $\langle \text{bid_id}, \text{cost}, \text{eta}, \text{reputation} \rangle$, or returning explicit refusals if overloaded.
3. **Bid Evaluation and Award:** The Manager evaluates bids according to a multi-attribute utility function $\mathcal{U}_M(b) = w_r \cdot \text{rep} - w_c \cdot \text{cost} - w_t \cdot \text{eta}$, awarding an exclusive worktree lease to the winning contractor and rejecting alternative bids.
4. **Execution, Interim Streaming, and Settlement:** The contractor checks out an isolated worktree branch, streams structured progress deltas, commits the diff, runs verification tests, and submits cryptographic execution receipts to unlock escrow payment.

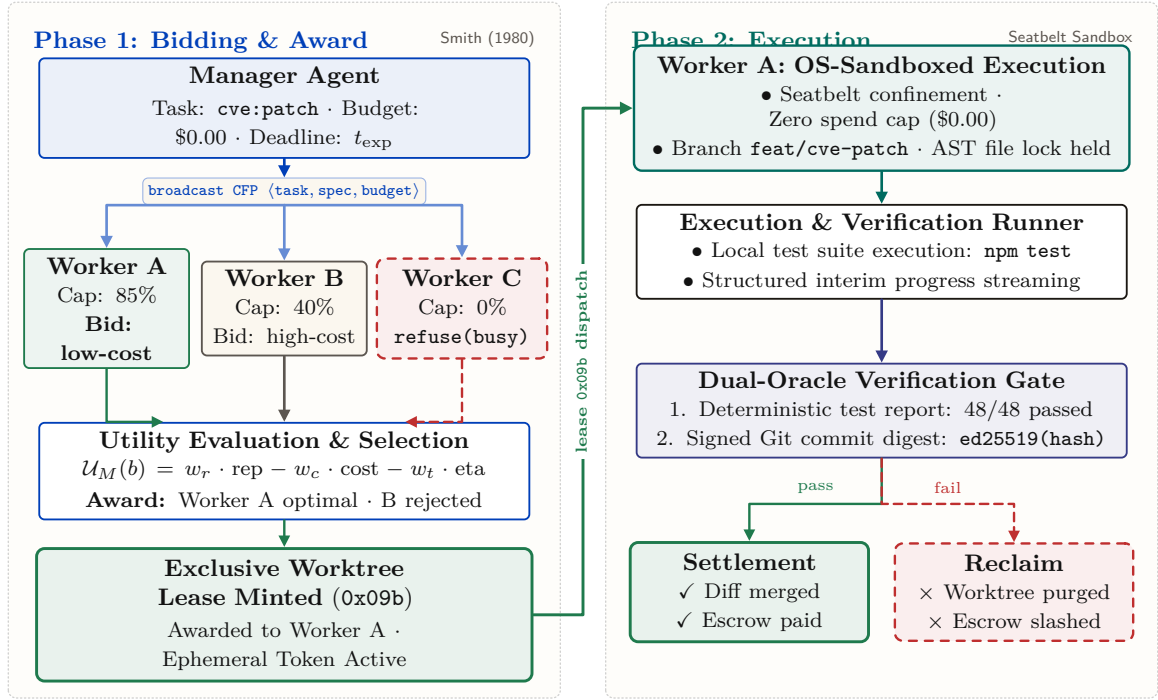
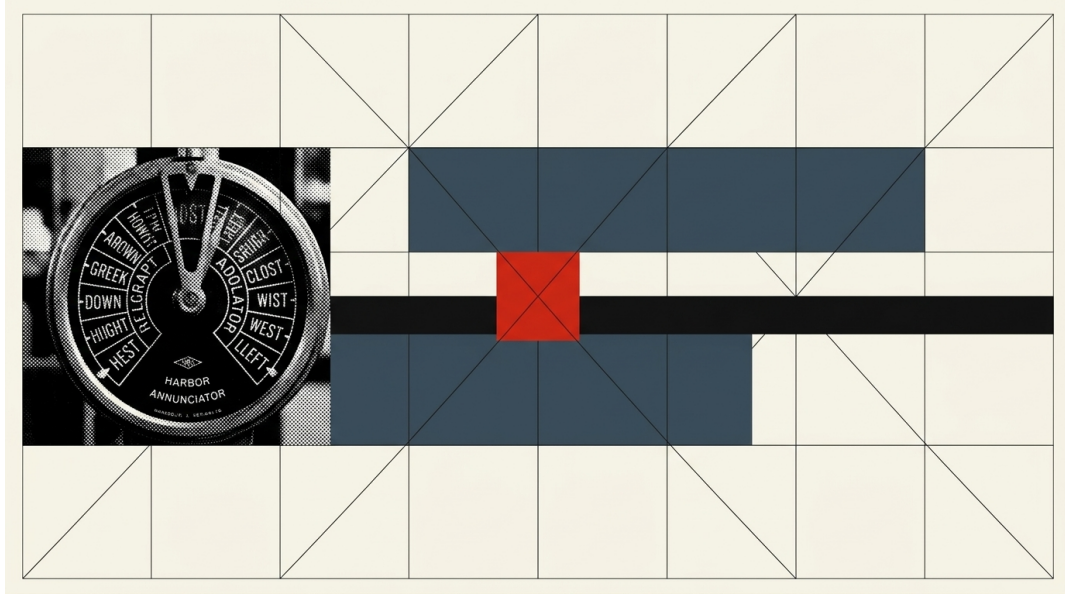


Figure 12: Contract Net Protocol (CNP) Task Allocation Lifecycle and Settlement Under Kernel Confinement. Grounded in Reid Smith’s (1980) bilateral market protocol. Phase 1 (left): Manager agent broadcasts a Call for Proposals (CFP) with task specification and spend cap; eligible contractors evaluate tool capacity and submit bids, or return explicit refusals under lock contention. Phase 2 (right): Winning contractor receives an exclusive worktree lease (0x09b), executing inside a Seatbelt sandbox under a zero-spend cap. Escrow funds unlock strictly upon presentation of signed git commit hashes and cryptographic test run receipts.



■ Architectural Plate 0.A · Task Allocation Dynamics (Contract Net Protocol)

“Decentralized market clearing replaces central bottlenecks; tasks are announced, evaluated against local capacity, and awarded under explicit capability leases.”

2.8.1 Why CNP Matters: Autonomous Task Allocation Without Central Dictators

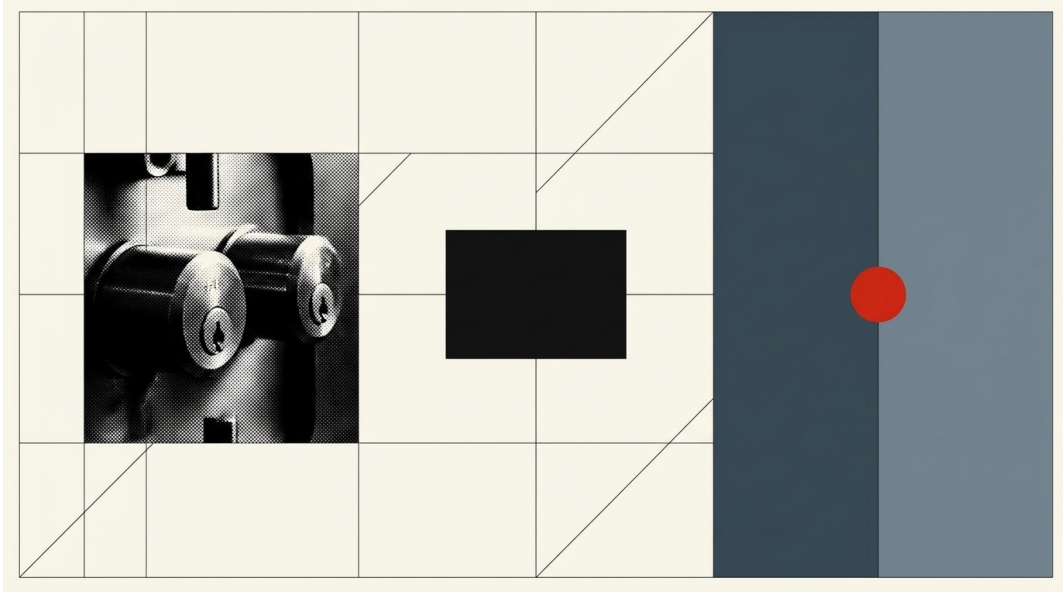
In any multi-agent engineering swarm, allocating tasks across workers presents a fundamental distributed systems dilemma:

1. **The Bottleneck of Central Planning:** If a centralized orchestrator agent must manually inspect each worker’s context window, parse their local git branches, and micromanage work assignment, the orchestrator becomes a severe throughput bottleneck and single point of cognitive failure.
2. **The Chaos of Greedy Contention:** Conversely, if workers greedily claim tasks from an uncoordinated queue, multiple agents race to modify the same files, squandering token budgets on overlapping AST diffs and inducing git merge conflicts.

Reid Smith’s Contract Net Protocol (CNP) [30] provides the decentralized market-clearing mechanism that solves this coordination dilemma:

- **What We Adopt (Bilateral Market Clearing):** We adopt CNP’s four-phase protocol: (1) Call for Proposals (CFP) broadcast with explicit task specifications and spend caps; (2) Capability self-assessment and bidding by candidate workers; (3) Exclusive lease award to the optimal bid; and (4) Escrow settlement upon verified delivery. In Port Daddy, this protocol directly powers autonomous dispatch (`pd dispatch`) and the Harbor Work Register: tasks are announced across the fleet, candidate subagents evaluate their own context capacity and tool availability before bidding, and the winning contractor is awarded an exclusive worktree branch.
- **What We Improve (Zero-Trust Confinement and Cryptographic Verification):** Classical 1980s CNP assumed honest, cooperative agents operating in academic simulation. In a real-world developer environment with stochastic LLMs, agents can suffer hallucinations, attempt

unauthenticated network egress, or introduce breaking bugs. We therefore harden CNP with modern systems security: the winning contractor executes strictly inside an isolated OS sandbox (macOS Seatbelt, Linux Landlock) under a strict spend cap (\$0.00 external spend). Crucially, settlement is dual-controlled: payment and task closure are released only upon cryptographic proof of passing test runs and signed git commit digests.



■ **Architectural Plate 0.B · Bilateral Escrow Settlement**

“Dual-control settlement guarantees that escrow payments unlock strictly upon cryptographic proof of passing test runs and signed git commit digests.

2.9 Dynamic Epistemic Logic and Big Brother Logic

In multi-agent swarms, coordination requires reasoning about higher-order knowledge: what agent i knows about what agent j knows. In modal epistemic logic ($S5_n$), Kripke relations R_i represent epistemic indistinguishability.

Public Announcement Logic (PAL) models communication as epistemic model restriction: when proposition ψ is announced, worlds where ψ is false are eliminated ($W' = \{w \in W \mid \mathcal{M}, w \models \psi\}$), establishing common knowledge $C_G\psi$.

Charrier et al. [11] developed **Big Brother Logic (BBL)**, formalizing epistemic reasoning over embodied agents equipped with directed observation cones (vision sets $V(a)$). If agent a_1 observes agent a_2 ($a_2 \in V(a_1)$), a_1 knows what a_2 observes, yielding $K_{a_1}K_{a_2}(\phi)$. By inverting verification into a planning problem, BBL proves that multi-agent systems can achieve target epistemic goals Φ by autonomously reconfiguring observation cones and network routes.

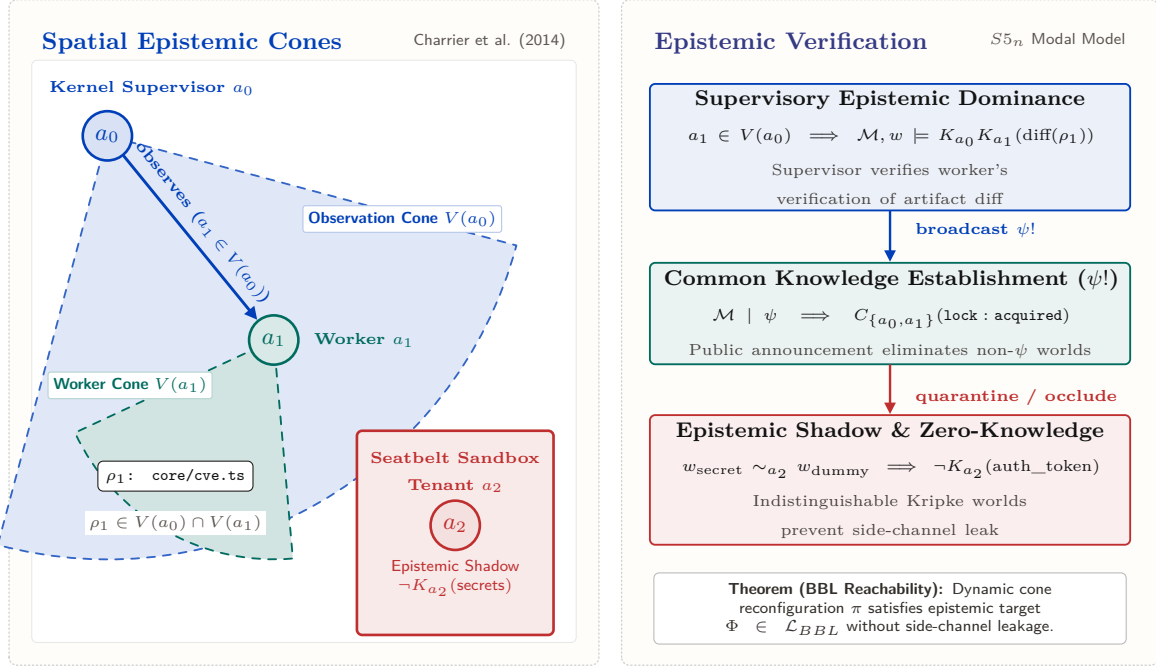
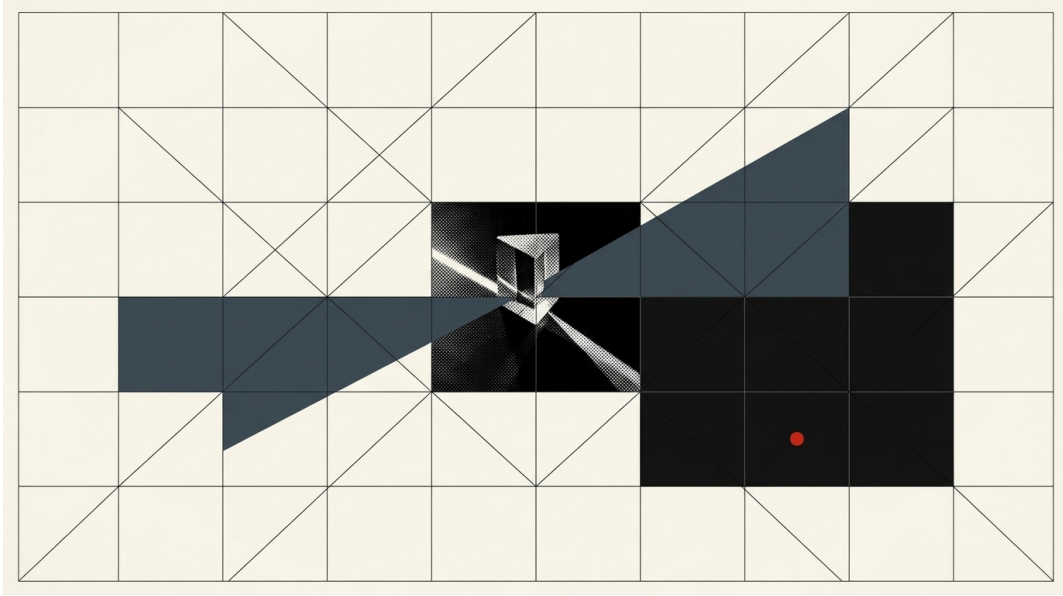


Figure 13: Dynamic Epistemic Observation Cones in Big Brother Logic (BBL) (Charrier et al., 2014). Directed vision cones formalize asymmetric visibility, epistemic shadows, and higher-order knowledge verification ($K_{a_1} K_{a_2} \phi$) in autonomous multi-agent systems. The supervisor maintains epistemic dominance over worker mutations, while sandboxed tenants operate within strict epistemic shadows, preventing side-channel data exfiltration.



■ Architectural Plate 0.C · Dynamic Epistemic Cones & Confinement

“Asymmetric observation cones guarantee supervisory dominance over worker mutations while confining tenants within strict epistemic shadows.”

2.9.1 Why Big Brother Logic Matters: Asymmetric Observation and Zero-Trust Confinement

In multi-agent systems research, information sharing is often naively treated as an unalloyed good: the more agents communicate, the better they coordinate. In production systems engineering, this assumption is catastrophic. Unrestricted visibility leaks proprietary credentials, inflates context windows with irrelevant chatter, and enables prompt-injection side-channels.

Charrier et al.’s Big Brother Logic [11] provides the formal epistemic apparatus required to model *asymmetric visibility* in security-critical environments:

- **The Supervisory Epistemic Invariant:** In an Agent Operating Kernel, the supervisor must maintain complete epistemic dominance over worker side effects ($K_{\text{kernel} \text{action}_i}$ holds for all admitted tool calls). Every command execution, file modification, and network attempt must fall strictly within the kernel’s directed observation cone.
- **Epistemic Shadows for Tenant Privacy:** Conversely, sandboxed workers must operate within strictly bounded epistemic shadows: worker a_i must not observe the private working memory, active credentials, or uncommitted diffs of peer worker a_j ($K_{a_i} \phi_j = \perp$). Furthermore, worker observation cones must never penetrate the host’s privileged secrets (e.g., system keychains, master API tokens).
- **What We Operationalize in Port Daddy:** We implement BBL’s observation cones through OS-level sandbox profiles (macOS Seatbelt, Linux Landlock). The kernel configures filesystem and network visibility cones dynamically per task: a security review agent is granted read-only visibility over repository source trees, but zero visibility over deployment credentials or local network sockets. By framing confinement as an epistemic geometry problem, the kernel guarantees that agents can collaborate effectively while strictly preserving organizational confidentiality boundaries.

2.10 Game-Theoretic Foundations and Mechanism Design

When agents have independent objective functions, coordination cannot assume altruism. Shoham and Leyton-Brown [29] systematized the algorithmic and game-theoretic foundations of multi-agent systems:

1. **Normal-Form Games and Nash Equilibrium:** An n -agent game is a tuple $\langle N, (A_i), (u_i) \rangle$. A strategy profile $s^* = (s_1^*, \dots, s_n^*)$ is a Nash equilibrium if no agent can unilaterally deviate and increase its expected payoff:

$$\forall i \in N, \quad \forall s_i \in S_i, \quad u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*) \quad (16)$$

2. **Mechanism Design and the Revelation Principle:** Mechanism design inverts game theory: given desired social outcomes (e.g., allocative efficiency, pareto optimality), the designer creates the game rules (action spaces and payment functions) such that rational agents truthfully reveal their private types θ_i . By the **Revelation Principle**, any social choice function implementable in Bayes–Nash or dominant-strategy equilibrium can be implemented by a *direct-revelation mechanism* where truth-telling is an equilibrium:

$$u_i(v_i(t_i, \theta_i) - p_i(t_i)) \geq u_i(v_i(\hat{\theta}_i, \theta_i) - p_i(\hat{\theta}_i)), \quad \forall \hat{\theta}_i \neq \theta_i \quad (17)$$

3. **Vickrey–Clarke–Groves (VCG) Mechanisms:** For allocating discrete resources (e.g., exclusive GPU reservation slots, compile worker nodes), VCG enforces dominant-strategy incentive compatibility (DSIC) by charging each agent the *externality* its presence imposes on all other agents:

$$p_i(\hat{\theta}) = \max_{k \in K} \sum_{j \neq i} \hat{v}_j(k) - \sum_{j \neq i} \hat{v}_j(k^*(\hat{\theta})) \quad (18)$$

Under VCG, truthful bidding ($\hat{\theta}_i = \theta_i$) is a strictly dominant strategy regardless of peer agent behavior.

2.11 Social Choice Theory and Arrow’s Impossibility Theorem

When a swarm of autonomous agents must aggregate individual preference orderings over candidate actions (e.g., selecting an architectural proposal, choosing a refactoring strategy), social choice theory governs the aggregation rules.

Let A be a finite set of alternatives and $N = \{1, \dots, n\}$ be the set of voting agents. Each agent i has a strict preference ordering $\succ_i \in \mathcal{L}(A)$. A **Social Welfare Function (SWF)** is a mapping:

$$F : \mathcal{L}(A)^n \rightarrow \mathcal{L}(A) \quad (19)$$

that maps a profile of individual rankings $(\succ_1, \dots, \succ_n)$ into a collective ranking $\succ$.

In 1951, Kenneth Arrow proved that no rank-order voting system can simultaneously satisfy four minimal criteria of fair aggregation [3]:

Theorem 2.2 (Arrow’s Impossibility Theorem (1951)). *If $|A| \geq 3$, any Social Welfare Function $F : \mathcal{L}(A)^n \rightarrow \mathcal{L}(A)$ that satisfies:*

1. **Unrestricted Domain (U):** *F is defined for all logically possible preference profiles.*
 2. **Weak Pareto (P):** *If $\forall i \in N, x \succ_i y$, then $x \succ y$.*
 3. **Independence of Irrelevant Alternatives (IIA):** *For any two alternatives x, y , the collective ranking between x and y depends solely on individual preferences between x and y .*
- is necessarily a Dictatorship (D): $\exists i \in N$ such that $\forall x, y \in A, x \succ_i y \implies x \succ y$.*

2.11.1 Why Arrow’s Theorem Matters: Why Multi-Agent Swarms Cannot Simply “Vote”

In naive multi-agent architectures, consensus is often implemented as a simple “agent vote”: three subagents generate proposed solutions, and a majority vote decides which code to commit. Arrow’s Impossibility Theorem [3] mathematically proves why this naive voting model is fatally flawed:

1. **Condorcet Cycles and Deadlock:** In the presence of three or more candidate solutions (A, B, C) , ordinal majority voting inevitably generates cyclic preference loops $(A \succ B \succ C \succ A)$. The swarm falls into an infinite deadlock, unable to pick a winner without arbitrarily breaking ties.
2. **Strategic Manipulation (Gibbard–Satterthwaite):** By the Gibbard–Satterthwaite theorem [14, 26], any non-dictatorial ordinal voting scheme is vulnerable to strategic misrepresentation: an agent can achieve a preferred outcome by dishonestly ranking candidates.
3. **The Inevitability of Dictatorship:** If an ordinal voting system eliminates cycles and satisfies Pareto efficiency, it mathematically degenerates into a dictatorship: one agent’s ranking unilaterally dictates the swarm’s decisions, rendering the remaining agents superfluous.

To circumvent Arrow’s impossibility, an Agent Operating Kernel must reject ordinal rank-order voting:

- **Cardinal Utility and Bonded Bidding:** Arrow’s theorem applies strictly to *ordinal* preference rankings (rank-order lists). By transitioning to *cardinal utility* backed by verifiable economic stake (bonded escrow tokens), the kernel bypasses Arrow’s conditions. Agents do not simply vote; they express bounded utility bids priced in verifiable compute and escrow balances.

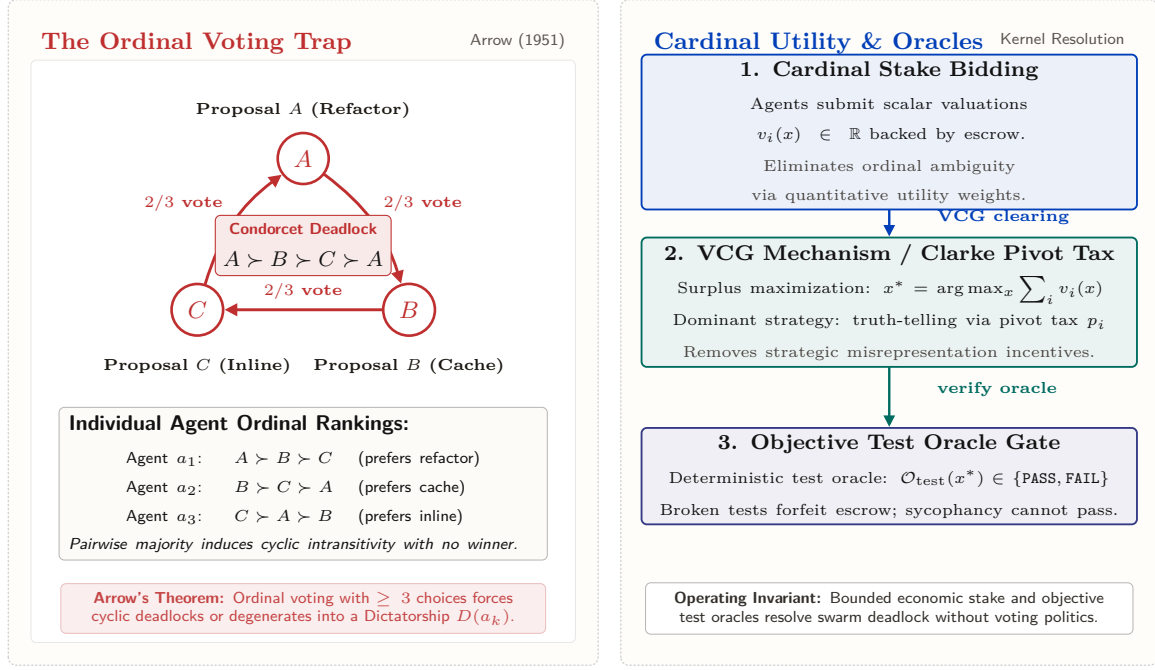


Figure 14: Arrow’s Impossibility Theorem in Autonomous Multi-Agent Consensus. Kenneth Arrow (1951) demonstrated that with $|A| \geq 3$, no ordinal preference aggregation function can simultaneously satisfy Unrestricted Domain (U), Weak Pareto (P), and Independence of Irrelevant Alternatives (IIA) without degenerating into a Dictatorship (D). In multi-agent systems, naive voting induces Condorcet cycles ($A \succ B \succ C \succ A$). Port Daddy avoids ordinal voting cycles by using cardinal utilities, bonded escrow, and VCG clearing.

- **Objective Test Oracles:** Crucially, code correctness is not a political consensus. A pull request that fails the test suite cannot be redeemed by a majority vote of three sycophantic language models. In Port Daddy, arbitration relies on objective deterministic test oracles and Clarke pivot taxes (VCG clearing), guaranteeing incentive compatibility without voting deadlocks.

2.12 The Ostrom Institutional Commons

When autonomous agents share developer workstation resources (TCP ports, CPU threads, disk space, git branches), they interact within a **Common-Pool Resource (CPR)**. Elinor Ostrom’s Nobel prize-winning institutional economics [21] disproved the dogma that common resources must inevitably suffer the “tragedy of the commons” or be privatized.

Ostrom established eight core design principles for sustainable, self-governing institutions:

1. **Clearly Defined Boundaries:** Clear demarcation of who has access to which resources.
2. **Congruence Between Appropriation Rules and Local Conditions:** Resource consumption rules match environmental scarcity.
3. **Collective-Choice Arrangements:** Individuals affected by operational rules can participate in modifying them.
4. **Active Monitoring of Resource Condition and User Behavior:** Monitors are accountable to users.
5. **Graduated Sanctions:** Violators face escalating penalties rather than binary expulsion.
6. **Low-Cost Conflict Resolution Mechanisms:** Rapid, local arbitration of disputes.

7. **Minimal Recognition of Rights to Organize:** External authorities do not override self-governance.
8. **Nested Enterprises:** Governance activities are organized in multiple layers of nested institutions.

In Chapter ??, we operationalize Ostrom’s eight principles into the **Bonded Commons**, transforming a fragile multi-agent development environment into a self-policing, economically resilient institution.

3 The Physics of Modern LLM Execution: What an Operating Kernel Must Know

To govern large language models, an operating system must understand the physical and computational realities of their execution. This section establishes the fundamental physics of modern LLMs: attention mechanics, memory bandwidth bottlenecks, training alignments, stochastic entropy, and context economics.

3.1 Transformer Mechanics: Self-Attention, Quadratic Complexity, and the KV-Cache

The universal engine of modern generative AI is the Transformer [44]. Let $X \in \mathbb{R}^{L \times d}$ be an input sequence of L tokens projected into a d -dimensional embedding space. In Multi-Head Attention (MHA), the input is mapped via learned weight matrices $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ into Queries (Q), Keys (K), and Values (V):

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \quad (20)$$

The core Scaled Dot-Product Attention operator is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (21)$$

3.1.1 The Quadratic Attention Wall

The matrix product QK^T forms an $L \times L$ attention matrix representing the pairwise affinity between every token in the sequence. Computing and storing this matrix imposes fundamental asymptotic costs:

$$\text{Compute} = O(L^2 \cdot d), \quad \text{Memory} = O(L^2 + L \cdot d) \quad (22)$$

While FlashAttention [36] uses GPU SRAM tiling to avoid materializing the full $L \times L$ matrix in High-Bandwidth Memory (HBM), the underlying FLOP count remains quadratic in context length L . For an agent maintaining an uncompact context of $L = 128,000$ tokens, a single attention pass requires evaluating $\approx 1.63 \times 10^{10}$ token-pair affinities per attention head.

3.1.2 The Key-Value (KV) Cache and Memory Bandwidth Bottlenecks

In autoregressive text generation, the model predicts one token at a time: $x_t \sim P_\theta(x_t \mid x_{<t})$. Because previous tokens do not change, recomputing their Key and Value representations at every decoding step would result in $O(L^3)$ aggregate compute. Production inference engines therefore cache Key and Value vectors in high-bandwidth GPU memory (HBM).

For a transformer with N_l layers, N_h key-value heads, dimension d_h , sequence length L , and precision b_p bytes (typically 2 for FP16/BF16), the KV-cache memory consumption per active agent sequence is:

$$\text{Memory}_{\text{KV}} = 2 \times N_l \times N_h \times d_h \times L \times b_p \quad \text{bytes} \quad (23)$$

Consider a frontier 70-billion parameter model ($N_l = 80$, $N_h = 8$ using Grouped-Query Attention, $d_h = 128$, $b_p = 2$). At context length $L = 128,000$:

$$\begin{aligned} \text{Memory}_{\text{KV}} &= 2 \times 80 \times 8 \times 128 \times 128,000 \times 2 \text{ bytes} \\ &\approx 41.94 \text{ GB per sequence} \end{aligned} \quad (24)$$

A fleet of 8 concurrent agent workers requires ≈ 335.5 GB of high-bandwidth memory solely to store the KV-cache, exceeding four 80 GB NVIDIA H100 GPUs before allocating a single byte for model weights.

3.1.3 Arithmetic Intensity: Compute-Bound Prefill vs. Memory-Bound Decoding

A critical physical reality of GPU execution is **Arithmetic Intensity**: the ratio of floating-point operations (FLOPs) to memory access (bytes transferred):

$$I = \frac{\text{FLOPs}}{\text{Bytes Transferred}} \quad (25)$$

Transformer execution is bifurcated into two radically distinct physical regimes:

1. **Prefill Phase (Prompt Processing)**: The prompt of length L_{prompt} is evaluated in parallel. Matrix multiplications (QK^T and AV) execute across all tokens concurrently, achieving high arithmetic intensity ($I \gg 100$ FLOPs/byte). Prefill is strictly **compute-bound**, saturating GPU tensor cores.
2. **Decoding Phase (Token Generation)**: Generation is strictly sequential: generating token x_{t+1} requires loading all model weights (~ 140 GB for 70B FP16) and the entire historical KV-cache from HBM to compute a single matrix-vector multiplication. The arithmetic intensity drops to $I \approx 1$ FLOP/byte. Decoding is strictly **memory-bandwidth bound**.

This physical bottleneck means that verbose agent prompts and long execution logs directly choke generation throughput. If an agent pollutes its prompt context with raw binary files or repetitive test output, it forces the inference server into massive memory-bandwidth stalls, driving latency from milliseconds to tens of seconds per token.

3.2 Training Regimes and the Alignment Illusion

Modern coding models undergo a three-stage training pipeline, each stage imparting distinct behavioral properties:

1. **Self-Supervised Pretraining**: The base model is trained on multi-trillion-token corpora of open-source repositories, documentation, and web data, optimizing next-token predictive likelihood:

$$\mathcal{L}_{\text{PT}}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}} \left[\sum_{t=1}^T \log P_{\theta}(x_t \mid x_{<t}) \right] \quad (26)$$

Pretraining instills syntactic fluency, programming language grammars, algorithmic patterns, and broad world knowledge. However, base models cannot follow instructions reliably; they simply predict statistically probable text continuations.

2. **Supervised Fine-Tuning (SFT) with Loss Masking:** SFT trains the model to act as an assistant, following instruction schemas and executing tool calls. Crucially, modern tool-use SFT employs **loss masking**: gradients are computed strictly over assistant tokens and tool call arguments, masking prompt templates and system instructions:

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{t \in \mathcal{T}_{\text{assistant}}} \log P_{\theta}(y_t \mid x, y_{<t}) \quad (27)$$

SFT enables models to emit valid JSON-Schema tool invocations and parse structured API responses.

3. **Preference Alignment (RLHF and DPO):** Reinforcement Learning from Human Feedback (RLHF) and Direct Preference Optimization (DPO) [43] align model outputs with human judgments of helpfulness, harmlessness, and conciseness:

$$\begin{aligned} \mathcal{L}_{\text{DPO}}(\theta; \theta_{\text{ref}}) = & -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} \right. \right. \\ & \left. \left. - \beta \log \frac{\pi_{\theta}(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)} \right) \right] \end{aligned} \quad (28)$$

3.2.1 The Alignment Trap: Conversational Compliance $\neq$ Execution Invariants

A fatal misconception among software engineers is assuming that an aligned model (trained via RLHF/DPO) will respect security boundaries and system rules. Alignment optimizes for *conversational agreeableness* and *plausible compliance*, not *formal invariant enforcement*.

When a human evaluator provides feedback during RLHF, an apology is rewarded as polite and constructive. Consequently, when an aligned agent encounters a permission denial or invariant failure, it reflexively generates contrite natural language: “*I apologize for the oversight. I will not modify that protected file again.*” But because autoregressive generation lacks physical state commitment, the model conditioned on its own apology has a non-zero probability of generating the exact same forbidden file mutation on the very next turn. Conversational alignment creates the dangerous illusion of safety while providing zero structural guarantees. True safety requires external, non-bypassable operating kernel gates.

3.3 Sampling Entropy, Temperature, and the Non-Determinism Hazard

During generation, logit vector $z \in \mathbb{R}^{|V|}$ is transformed into a token probability distribution via softmax with temperature parameter $T \geq 0$:

$$P(w_i \mid x_{<t}) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \quad (29)$$

To prevent degenerate text loops while truncating low-probability tail tokens, inference engines apply **Nucleus (Top- p) Sampling**:

$$V^{(p)} = \left\{ w \in V \mid \sum_{w' \in V, P(w') \geq P(w)} P(w') \leq p \right\} \quad (30)$$

Theorem 3.1 (Non-Determinism of Autonomous Execution). *Let $\mathcal{I}$ be a safety invariant over execution environment $\mathcal{S}$. For any autoregressive policy $\pi_{\theta}(a_t \mid h_t)$ evaluated at temperature $T > 0$, the single-step probability of generating an invariant-violating action $a_t \notin \mathcal{A}_{\text{safe}}$ is strictly positive:*

$$\exists \epsilon > 0 \quad \text{such that} \quad \Pr[a_t \notin \mathcal{A}_{\text{safe}} \mid h_t] \geq \epsilon \quad (31)$$

Consequently, over an autonomous multi-turn workflow of length N , the cumulative probability of safe execution decays exponentially:

$$\Pr[\text{Safe Execution over } N \text{ steps}] \leq (1 - \epsilon)^N \xrightarrow{N \rightarrow \infty} 0 \quad (32)$$

Theorem 3.1 proves that an operating system cannot rely on prompt instructions or model self-policing for invariant preservation. As an agent’s task length N increases, catastrophic failure is mathematically guaranteed. Furthermore, even at $T = 0$ (greedy decoding), non-deterministic floating-point reduction order across GPU CUDA threads can yield divergent token traces for identical prompts. System determinism must therefore be supplied entirely by the external host kernel.

3.4 The Context Window as Volatile, Lossy RAM

The transformer context window is frequently analogized to computer RAM. This analogy is profoundly misleading: computer RAM is byte-addressable, persistent until power loss, and incurs $O(1)$ read latency. The context window, by contrast, is volatile, lossy, and subject to severe attention degradation.

3.4.1 The “Lost in the Middle” Phenomenon

Liu et al. [19] demonstrated that retrieval performance in long-context models follows a severe U-shaped curve:

- **Primacy Effect:** Tokens placed at the beginning of the context (system instructions, core role definitions) receive high attention weights.
- **Recency Effect:** Tokens placed at the immediate end of the context (latest turn, most recent tool outputs) receive high attention weights.
- **Attention Dilution in the Trough:** Facts, schemas, and constraints situated in the middle 60% of the context window suffer drastic retrieval degradation, with accuracy dropping by up to 70%.

If an agent harness maintains state simply by appending intermediate execution traces, crucial security constraints and AST symbols migrate into the middle trough, where the model effectively forgets them.

3.4.2 Context Compaction Lossiness and the Necessity of Externalized State

When a multi-turn conversation approaches context capacity, harnesses perform context compaction: summarizing historical turns to free token budget. In-context summarization is fundamentally lossy:

- Compaction drops exact file paths, compiler error codes, and git commit hashes.
- Hallucinations during compaction rewrite history: an agent that previously failed a test may summarize the turn as “verified functionality,” permanently corrupting its belief state.
- Uncommitted diffs and open capability leases are forgotten, inducing resource leaks and zombie sessions.

Design Invariant 3.2 (Externalized Durable State).

An agent operating kernel must never rely on the model’s context window as the system of record. All locks, active capabilities, file claims, financial ledgers, and execution proofs must be externalized into an out-of-band, ACID-compliant database (SQLite WAL), with prompt buffers containing only temporary, ephemeral projections.

4 Modern Coding Agent Architectures, Planning, and Search

We now examine how modern LLM-based coding agents interface with host environments, structure deliberation, and execute search over complex problem spaces.

4.1 The Contemporary Landscape: Systems Comparison

Contemporary coding agents bridge unstructured natural language reasoning with deterministic host tools (shells, AST parsers, compilers, git checkouts). Table 4 compares the leading production architectures:

Agent	Isolation Boundary	Context Strategy	Tool Interface	Rollback Engine
Claude Code	Subprocess + Seatbelt	Progressive disclosure	File edit, bash, MCP	Git commit checkpoint
Codex CLI	Container sandbox	History compaction	Sandboxed patch apply	Ephemeral container reset
Gemini CLI	Local / Cloud Run	1M–2M token window	POSIX CLI, GCP SDK	Workspace rewind
Aider	Host process	Repo-map (Tree-sitter)	Git-backed diff engine	Automatic git commit undo
Devin	MicroVM (Firecracker)	Multi-buffer browser/shell	Full root pseudo-terminal	Snapshot restoration

Table 4: Architectural comparison of contemporary production coding agents.

4.2 Closed-Loop Deliberation vs. Open-Loop Sampling (POMDP)

Stateless code generation evaluates an autoregressive distribution $Y \sim P_\theta(Y \mid X)$ in an open loop. Any syntactic, semantic, or environmental error compounds irreversibly:

$$\text{ErrorRate}_{\text{open-loop}}(T) = 1 - \prod_{t=1}^T (1 - \epsilon_t) \quad (33)$$

In contrast, an agent harness operates as a Partially Observable Markov Decision Process (POMDP):

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \Omega, \mathcal{O}, \gamma \rangle \quad (34)$$

where $\mathcal{S}$ is the true environment state (filesystem, git index, running processes), $\mathcal{A}$ is the action space (tool invocations), Ω is the observation space (stdout, stderr, exit codes), and $\mathcal{O}$ generates observations. The agent maintains a belief state $b_t = \Pr(s_t \mid h_t)$ conditioned on history $h_t = (a_0, o_0, \dots, a_{t-1}, o_{t-1})$.

4.3 Deliberation, Planning, and Search Frameworks

Modern agents employ structured search and planning algorithms across candidate execution paths [47]:

1. **ReAct (Reasoning and Acting) [32]:** Interleaves natural language reasoning traces (“Thoughts”) with structured tool executions (“Actions”) and environment feedback (“Observations”). When factual uncertainty is encountered, reasoning halts and an exploratory tool is invoked.
2. **Tree of Thoughts (ToT) [33]:** Generalizes chain-of-thought prompting to tree-search algorithms (BFS, DFS, A*). The agent generates candidate reasoning steps, evaluates node value via self-critique heuristics, and backtracks upon exploring unviable subtrees.
3. **Reflexion [28]:** Implements verbal reinforcement learning. Upon test failure or verification rejection, the model generates an explicit post-mortem reflection analyzing the cause of error, which is persisted in an episodic memory buffer and injected into future prompt turns.
4. **Voyager Lifelong Skill Learning [45]:** Implements open-ended lifelong learning by accumulating an **executable skill library**. When an agent solves a complex programming task,

it synthesizes a self-contained, parameter-validated script and stores it in a vector-indexed codebase. Subsequent tasks retrieve and compose existing skills, transforming complex multi-step reasoning into atomic tool calls.

5. **TDAG: Topological Task DAG Execution [46]:** Formalizes task planning as a Directed Acyclic Graph (DAG) of interdependent subgoals. TDAG dynamically decomposes complex software workflows into parallel execution chains, resolves circular dependencies at runtime, and linearizes task dispatch across specialized worker subagents.
6. **Hypertree Planning [34]:** Hierarchical task network planning that recursively decomposes macro-objectives (e.g., “implement OAuth authentication”) into hyperedges connecting subgoals, preventing agent fleets from becoming trapped in localized code modifications.

4.4 Cognitive Control, Test-Time Compute, and Process Reward Models

Frontier coding models increasingly shift computational budget from training to inference time. Instead of sampling greedily from $\pi_\theta(a | h)$, inference search trees explore alternative reasoning chains scored by **Process Reward Models (PRMs)** that evaluate correctness at each logical derivation step:

$$\text{Score}(\tau) = \prod_{t=1}^T \text{PRM}(c_t | c_{<t}, x) \quad (35)$$

When an intermediate verification score falls below a threshold τ_{cut} , the search beam backtracks to a previous checkpoint.

4.5 Multi-Agent Formal Verification and Benchmarking

Evaluating multi-agent software engineering requires moving beyond synthetic static benchmarks (such as HumanEval) toward interactive, sandboxed evaluation suites:

- **AgentBench [41]:** A multi-dimensional benchmark evaluating LLMs as autonomous agents across operating system bash environments, databases, web browsing, and multi-agent coordination games.
- **HELM (Holistic Evaluation of Language Models) [40]:** Standardized multi-metric evaluation across accuracy, calibration, robustness, fairness, and toxicity.
- **FormalJudge (Zhou et al., 2026) [48]:** Combines neural LLM proposals with symbolic SMT solvers (Z3) and model checkers to verify formal code invariants at compile time, eliminating hallucinated test assertions.

5 Tools, Protocols, Skills, and Hooks

To interact safely with the host environment, an agent harness must govern tool dispatch, encapsulate domain knowledge, and enforce runtime invariants.

5.1 Model Context Protocol (MCP) as the Substrate

Anthropic’s **Model Context Protocol (MCP)** standardizes client-server interoperability between AI harnesses and host execution environments:

1. **Tools:** Executable remote procedures invoked by the agent (`tools/call`), accepting JSONSchema parameters and returning content blocks.
2. **Resources:** Passive data endpoints (`resources/read`) exposing context such as local file contents, system logs, or git diffs.
3. **Prompts:** Reusable prompt templates parameterizing multi-step workflows.

5.2 Context Economics and Progressive Skill Disclosure

A major vulnerability of early tool-use architectures was schema bloat: injecting hundreds of complex tool definitions into the prompt exhausted the context window before execution began. Modern agent harnesses resolve this dilemma via **Three-Tier Progressive Disclosure**:

1. **Tier 1 (Surface Frontmatter Index):** The system prompt contains only lightweight index entries: the skill’s unique identifier and a concise 1-to-2 sentence description (10^1 – 10^2 tokens per skill). Across a library of 500 skills, this consumes a static footprint of $\approx 15,000$ tokens ($\approx 7.5\%$ of a 200k context window). Because Tier 1 is static across all turns, modern inference providers cache this prompt prefix in HBM, reducing input costs by up to 90%. The probability of inclusion in the prompt is $P = 1.0$.
2. **Tier 2 (On-Demand Instruction Activation):** When the agent’s semantic classifier or planning policy determines that a task requires a specific capability, it invokes a read tool to load the corresponding `SKILL.md` into its working context (10^3 – 10^4 tokens). Tier 2 contains operational instructions, parameter constraints, decision tables, and error recovery protocols. Its activation probability is conditionally sparse ($P \approx 10^{-1}$), loaded only for the duration of the subtask and purged or compacted when the goal is achieved.
3. **Tier 3 (Deep Reference and Subprocess Execution):** Beneath the instruction manual lies the deepest architectural layer: specialized executable scripts (`scripts/`), schemas, and large reference manuals (`references/`), spanning 10^4 – 10^5 tokens. Crucially, the agent *does not load Tier 3 assets into its language context*. Instead, Tier 2 instructs the agent to execute Tier 3 scripts in an isolated host subprocess or query specific byte offsets on demand. Because invoking a specialized script or deep reference is needed only for esoteric domain edge-cases, its execution probability is orders of magnitude lower ($P \approx 10^{-3}$ to 10^{-4}).

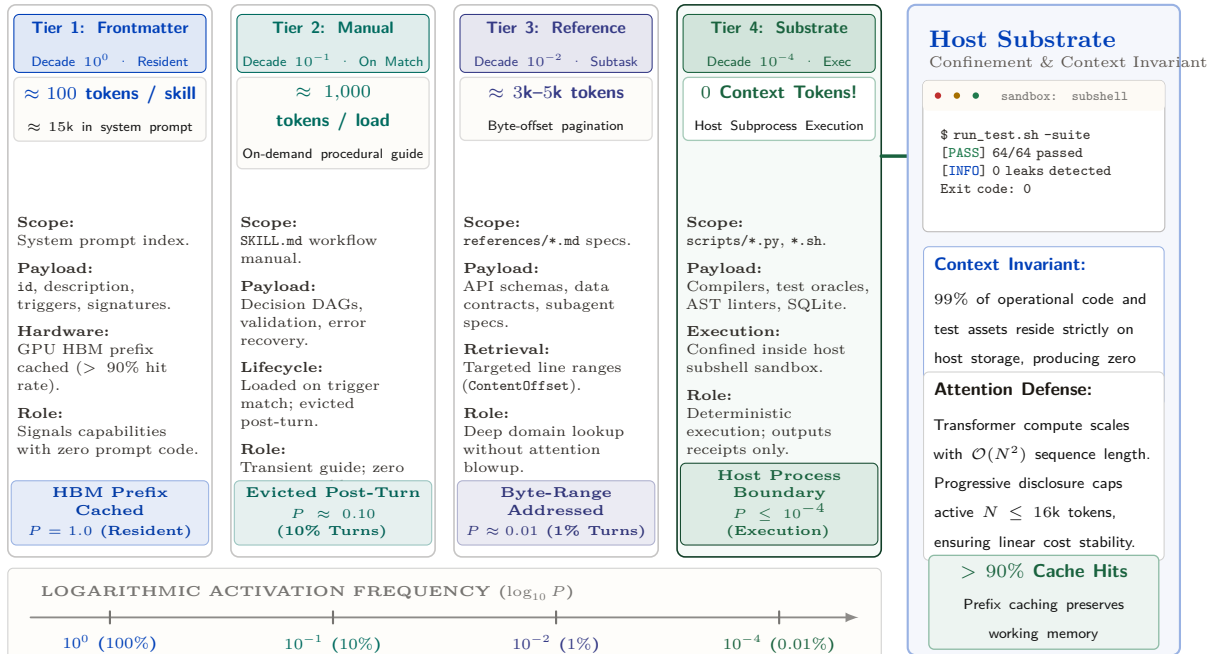
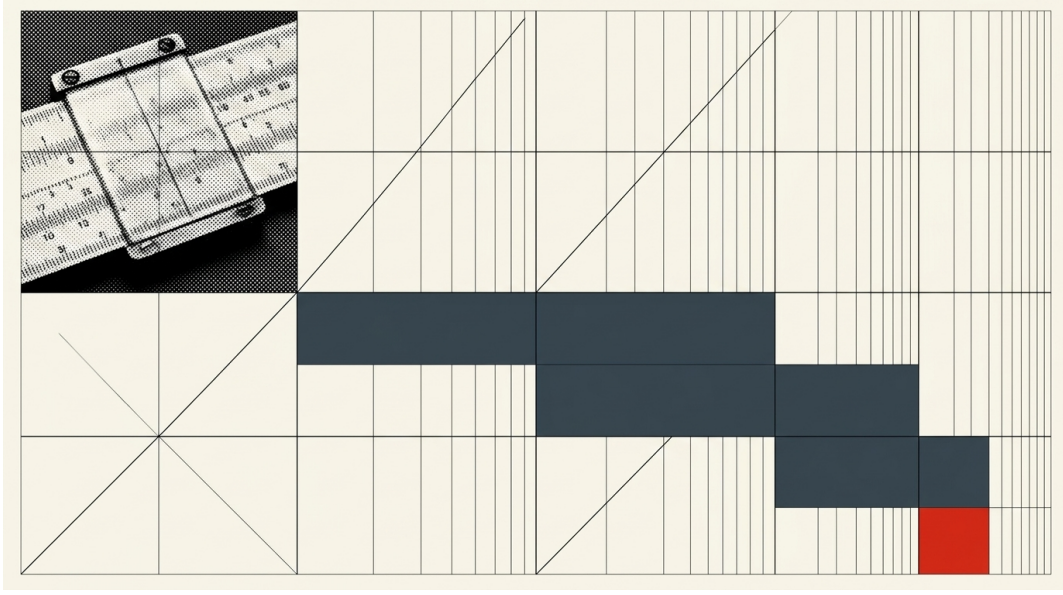


Figure 15: Progressive Disclosure of Skills Across Contracting Logarithmic Decades. Capability activation frequency contracts across decades (10^0 to 10^{-4}). By partitioning capabilities from static frontmatter metadata down to host-executed scripts and formal schemas consuming zero LLM context tokens, context consumption contracts exponentially with task specialization.



■ **Architectural Plate 0.D · Progressive Skill Disclosure Economics**

“Contracting logarithmic tiers of capability activation preserve working memory, executing deep scripts on host substrates with zero token footprint.”

5.3 Lifecycle Hooks and Supervisory Control

Harness lifecycle hooks intercept agent control flow at critical execution boundaries. Formally, lifecycle hooks operate as a **Ramadge–Wonham Supervisory Controller** (S/G) [22].

Let unconstrained agent execution be modeled as a discrete-event generator $G = \langle X, \Sigma, \delta, x_0, X_m \rangle$, where events $\Sigma = \Sigma_c \cup \Sigma_u$ are partitioned into controllable events Σ_c (tool invocations, file writes, network requests) and uncontrollable events Σ_u (model token generation, host hardware interrupts, external process termination). A supervisory hook $S : L(G) \rightarrow \mathcal{P}(\Sigma)$ dynamically disables tool invocations that violate safety invariants:

$$L(S/G) \subseteq K \subset L(G) \quad (36)$$

where K is the formal language of safe execution traces (e.g., preventing writes to protected branches, restricting filesystem mutations to leased workspaces, or halting when spending caps are exceeded).

Figure 16 formalizes the complete cross-section of modern production agent lifecycle hooks, modeled after the production architecture of Anthropic’s Claude Code and the Port Daddy Giant Squid Harness. Execution proceeds through two nested supervisory loops:

1. **The Outer Session & Turn Loop:** Encompasses initial context injection (**SessionStart**), prompt interception (**UserPromptSubmit**), context pressure management (**PreCompact**), turn-level invariant verification (**Stop/StopFailure**), and session teardown (**SessionEnd**).
2. **The Inner Agentic Tool Loop:** Governs the iterative reasoning-action cycle: pre-tool interception and deterministic vetoes (**PreToolUse**), sandboxed execution, and reactive post-tool sanitization (**PostToolUse**).

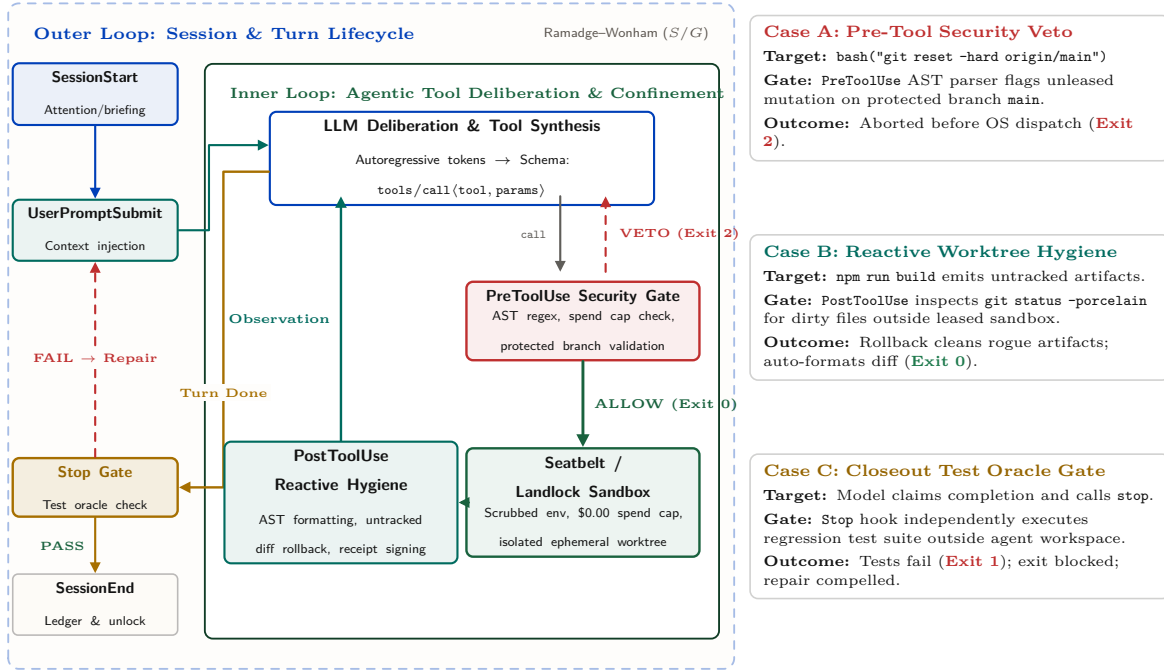


Figure 16: The Agent Lifecycle Hook Cross-Section. Modeled after the production architecture of Anthropic’s Claude Code and the Port Daddy Giant Squid Harness. Execution is partitioned into two nested supervisory loops: the outer Session and Turn lifecycle manages user context, memory compaction (**PreCompact**), and test verification (**Stop**), while the inner **AGENTIC TOOL LOOP** supervises iterative tool invocations, pre-execution security vetoes (**PreToolUse**), sandbox execution, and reactive post-tool sanitization (**PostToolUse**). Concrete production traces (Cases A–C) illustrate deterministic enforcement at binary interception points.

5.3.1 Case Studies in Lifecycle Hook Enforcement

In production harnesses, lifecycle hooks replace conversational model instructions with deterministic binary gates. The following three scenarios illustrate how the inner and outer supervisory loops enforce system invariants:

Case A (Pre-Tool Security Veto): *Destructive Command Interception.* An agent attempting to resolve a git merge conflict synthesizes a destructive tool invocation:

```
bash("git checkout main && git reset --hard origin/main").
```

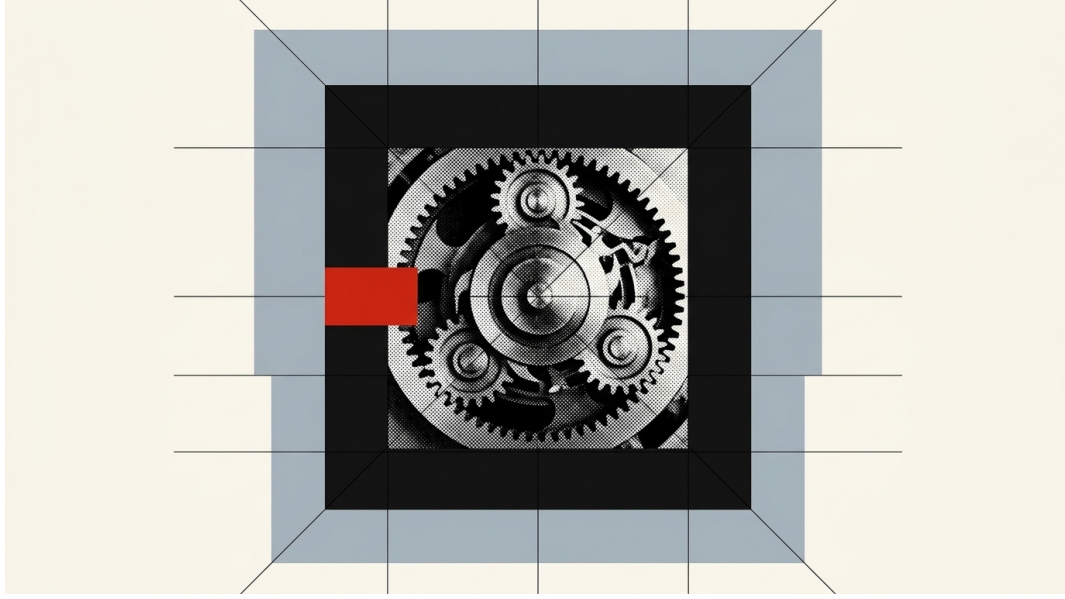
Before the command reaches the operating system shell, the `PreToolUse` hook executes an AST regex and flag parser. Detecting an unleased mutation targeting the protected branch `main`, the supervisor aborts the syscall with exit code 1, returning an explicit veto envelope:

```
<status : REJECTED, reason : "Direct destructive reset of main branch forbidden. Work must occur on leas
```

The destructive act is prevented at the effect boundary; the workspace remains pristine.

Case B (Post-Tool Diff Analysis): *Unintended Mutation Rollback.* An agent executes a build script ("`npm run build`") that inadvertently compiles release binaries into the source tree, mutating untracked files outside its leased directory. Upon process termination, the `PostToolUse` hook executes `git status --porcelain`. Detecting untracked modifications in forbidden paths, the hook automatically invokes a sandbox rollback, cleans the unleased artifacts, and injects a sanitized warning into the prompt context.

Case C (Pre-Compact Memory Preservation): *Critical Invariant Retention.* During a 40-turn refactoring session, context window consumption crosses 85% capacity, triggering the outer loop’s `PreCompact` hook. Rather than allowing naive model summarization to discard crucial diagnostic data, the hook intercepts the compaction routine. It extracts active symbol claims, open file locks, failing unit test stack traces, and active capability tokens, serializing them into an immutable structured JSON header that is prepended to the compacted summary.



■ Architectural Plate 0.E · Dual-Loop Supervisory Interception

“Deterministic lifecycle hooks intercept unconstrained generation at binary boundaries, replacing conversational advice with discrete supervisory vetoes.”

6 Multi-Agent Swarm Topologies, Coordination, and Multi-Agent Economics

When scaling from a single agent to a collaborative collective, coordination must transition from direct peer-to-peer communication to structured environmental and institutional governance.

6.1 Stigmergic Communication: Environmental Coordination

Why do multi-agent chat systems fail to scale past three or four concurrent participants? The answer lies in the physics of communication channels. In 1959, French biologist Pierre-Paul Grassé coined the term **Stigmergy** (from the Greek *stigma*, mark, and *ergon*, work) to explain how social insects construct intricate, cathedral-like termite mounds without central leadership or point-to-point communication.

In a stigmergic coordination architecture, agents do not broadcast conversational natural-language messages to one another ($O(N^2)$ communication complexity). Instead, **the environment itself is the communication medium**:

1. **Passive Environmental Modification:** Agents coordinate by leaving persistent, machine-readable traces directly in shared host artifacts: file locks, AST reservation tags, pheromone heat maps, and append-only note streams.
2. **Decoupled Sensing:** When Agent *A* claims an AST symbol or edits a function, it writes a tuple into the shared kernel tuple space. When Agent *B* inspects the repository, it senses Agent *A*’s trace and autonomously diverts its own planning policy away from the contested file.

Stigmergy eliminates conversational token saturation, reduces communication complexity from $O(N^2)$ to $O(N)$, and decouples agent execution in time.

6.2 Conway’s Law in Multi-Agent Swarms and Role-Based SOPs

In 1968, computer scientist Melvin Conway formulated the foundational law of software architecture [35]:

“Organizations which design systems are constrained to produce designs which are copies of the communication structures of these organizations.”

In multi-agent engineering swarms, Conway’s Law holds with absolute mathematical force:

- **The Broadcast Chat Pathology:** If a swarm communicates via unconstrained broadcast chat (every agent talking to every other agent), the resulting software architecture is inevitably an unmaintainable, tightly coupled, circular-dependent monolith.
- **MetaGPT Role-Based Standard Operating Procedures (SOPs) [37]:** Hong et al. demonstrated that multi-agent swarms achieve dramatic error reduction by encoding human software engineering Standard Operating Procedures into strictly partitioned roles: Product Managers generate PRDs, Architects synthesize system design diagrams, Engineers write modular code, and QA Reviewers run test suites. Communication flows unidirectionally through structured documents (PRDs, OpenAPI specs, git diffs) rather than unconstrained chat.

6.3 Six Sigma Defect Minimization in Agent Swarms

Huang et al. (2026) [38] demonstrated that as autonomous multi-agent pipelines scale across dozens of sequential tool steps, defect rates compound exponentially:

$$\text{Yield}_{\text{total}} = \prod_{k=1}^K (1 - \text{DefectRate}_k) \quad (37)$$

To achieve high process yield, swarms must implement **Six Sigma error-dampening gates**: every handoff between subagents must be verified by a deterministic oracle (compiler check, static analyzer, or formal test suite) before work is admitted to downstream stages. An unverified handoff is rejected back to the upstream agent, preventing errors from propagating through the swarm.

6.4 Runtime Task Overlap Detection, Solely Responsible Agent Patterns, and Sandboxing

To eliminate race conditions across concurrent workers, an operating kernel enforces three structural invariants:

1. **Runtime Task Overlap Detection:** The kernel continuously monitors AST-level symbol claims and active file reservations. If two agents attempt to claim overlapping function definitions or files, the conflict is detected instantly at wire-speed, granting an exclusive lease to the first claimant and refusing the second with an explicit retry backoff.
2. **Solely Responsible Agent (SRA) Patterns:** For every critical resource (a database migration script, a deployment configuration), exactly one agent is designated as solely responsible. Auxiliary subagents can submit proposals or review diffs, but only the SRA holds cryptographic authority to execute the final mutation.
3. **Confidential Local LLM Sandboxing:** Untrusted or external subagents execute in confined environments with stripped environment variables (scrubbing master API tokens, SSH keys, and cloud credentials), communicating strictly across local domain sockets under seat-belt confinement.

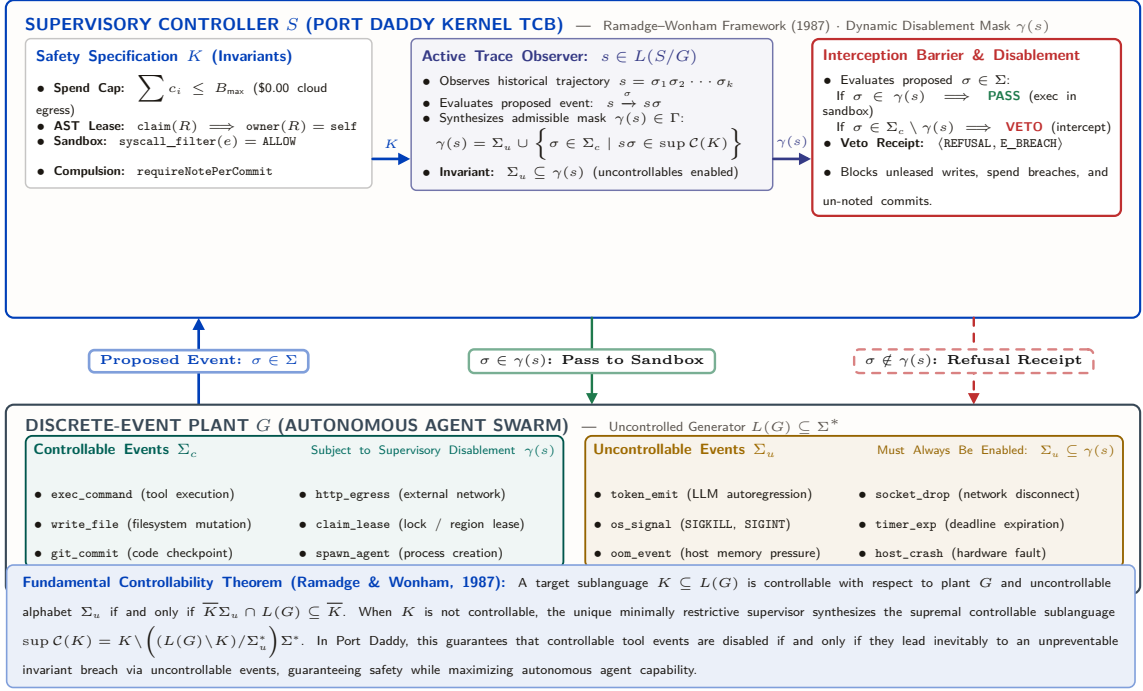


Figure 17: Ramadge–Wonham Discrete-Event Supervisory Control in Multi-Agent Swarms. Uncontrolled swarm execution G is restricted by supervisory controller S to enforce safe language $K \subset L(G)$. Controllable tool events Σ_c (file writes, process spawns, network requests) that violate mutual exclusion or spend caps are disabled by the supervisor, guaranteeing system-wide safety invariants.

6.5 Zombie Detection, Heartbeats, and State Salvage

In autonomous fleets, process death is a constant reality. An agent container may crash, freeze on an infinite loop, or be terminated by an OS OOM killer. If the kernel does not actively monitor liveness, abandoned file reservations and port claims become permanent locks, paralyzing the swarm.

The kernel implements a **Zombie Salvage Protocol**:

1. **Periodic Heartbeats:** Every active agent must post a signed heartbeat $\langle \text{agent_id}, t_{\text{now}} \rangle$ at regular intervals Δt_{hb} (typically 30 seconds).
2. **Liveness Watchdog and Eviction:** If an agent’s heartbeat lapses beyond a threshold $T_{\text{zombie}} = 3 \times \Delta t_{\text{hb}}$, the kernel marks the agent as a **Zombie**, revokes its active capability leases, and releases its file locks.
3. **Crash Salvage Ledger:** Crucially, the zombie’s work is not lost. The kernel inspects the deceased agent’s uncommitted worktree, harvests modified diffs and test logs into an append-only salvage ledger, and queues the task for re-allocation via Contract Net Protocol.

6.6 Closed-Loop Attestation and Execution Verification

Figure 18 illustrates the closed agentic loop, stratified into three architectural tiers: the probabilistic Deliberation Layer, the confined Execution Layer, and the deterministic Governance Layer.

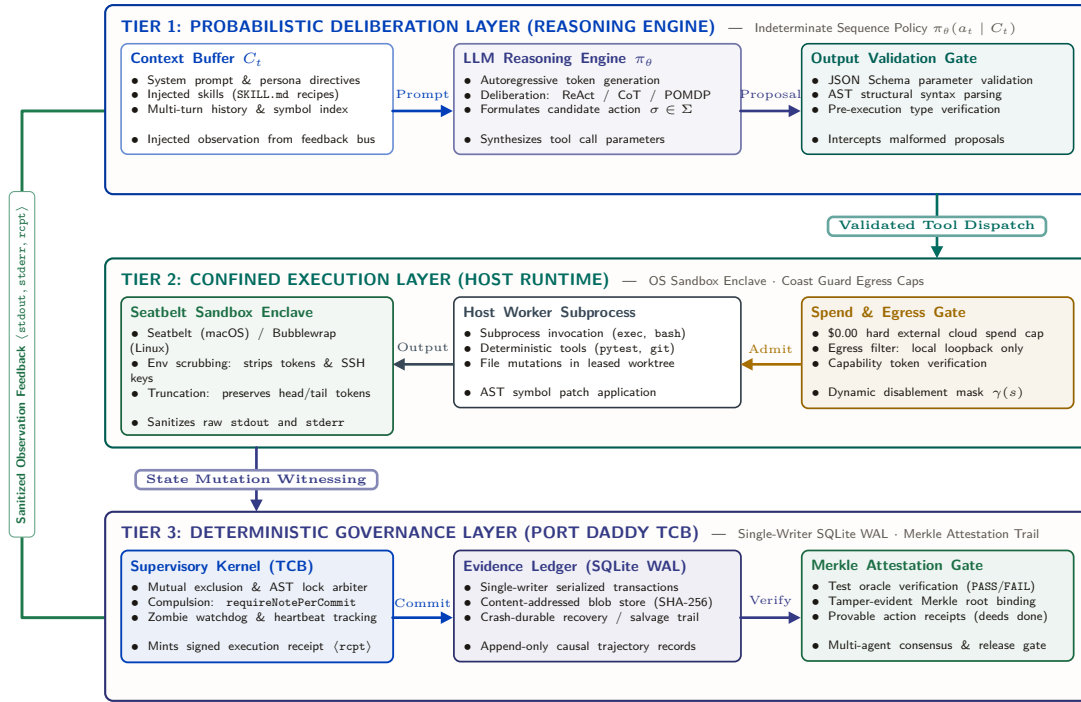
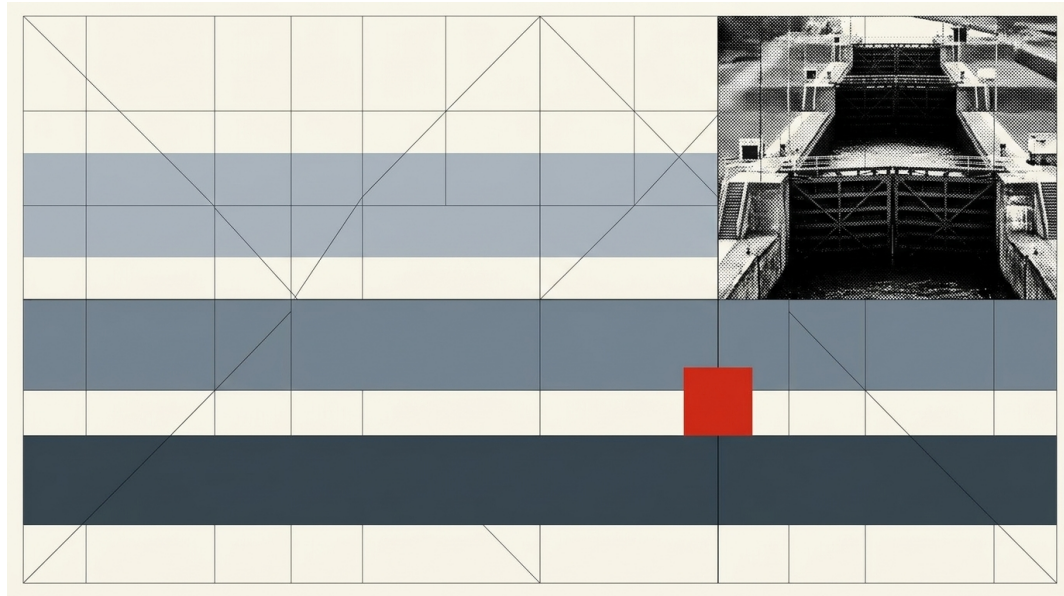


Figure 18: The Closed Agentic Architecture Loop. Three architectural tiers separating probabilistic deliberation from deterministic kernel coordination: (1) Probabilistic Deliberation Layer (LLM reasoning and planning); (2) Confined Execution Layer (sandboxed host containers and tool execution); and (3) Deterministic Governance Layer (single-writer SQLite WAL, capability verification, and tamper-evident Merkle attestation).



■ Architectural Plate 0.F · Closed-Loop Attestation & Governance

“Stratifying probabilistic deliberation from deterministic kernel attestation provides the authoritative bedrock for autonomous software engineering.

The harness operates as a deterministic finite-state machine driving an indeterminate generator:

1. **Pre-Invocation Assembly:** The harness compiles system instructions, tool schemas, retrieved repository symbols, and recent conversation turns into the token buffer.
2. **Inference:** The prompt is transmitted to the inference engine. Tokens stream autoregressively until a function-call termination token is detected.
3. **Validation and Policy Gate:** The parser extracts tool arguments. The supervisor validates types against JSON-Schema definitions and verifies authority against local capability policies.
4. **Sandboxed Execution:** The command executes inside an isolated runtime container (macOS Seatbelt, Linux Bubblewrap) under strict egress caps.
5. **Kernel Witnessing:** Mutations, locks, and file reservations are submitted to the local kernel, appending a receipt to the audit log.
6. **Observation Injection:** Raw output is sanitized, truncated to preserve token budget, and appended to the context buffer for the next deliberation step.

7 The Kernel Gap: Why LLMs Cannot Be Their Own OS

We conclude this foundational survey by establishing the formal impossibility theorem that motivates the entire volume:

Theorem 7.1 (Non-Sovereignty of Autoregressive Generation).

Let $\mathcal{M}$ be a generative autoregressive language model and let $\mathcal{S}$ be a shared physical environment state space subject to concurrent mutation. For any sequence policy $\pi_\theta(a_t \mid C_t)$ operating over context buffer C_t :

1. **Advisory Boundary:** The probability of invariant violation under open-loop generation is strictly positive: $\Pr[\text{violate}(\mathcal{I}) \mid \pi_\theta] > 0$, rendering prompt-level safety constraints insufficient for kernel-level security guarantees.
2. **Mutual Exclusion Incompleteness:** In the absence of an out-of-band serialized physical mutex $\mathcal{K}_{\text{TCB}}$, concurrent independent autoregressive streams π^A, π^B cannot guarantee serializability: $\exists \tau : \text{LostUpdate}(\tau) = \text{true}$.
3. **Amnesiac Irresponsibility:** The process state $\mathcal{M}_t$ vanishes upon SIGKILL or context eviction. Liability, capability attenuation, and resource conservation cannot be maintained endogenously by the model, but must be enforced by an external, crash-durable supervisory kernel.

Therefore, generative models cannot act as their own operating system kernel; an authoritative, out-of-band coordination substrate is strictly required.

8 Chapter Map and Architectural Codas

8.1 Structure of the Volume

The remainder of this textbook is organized into eight cumulative chapters across four distinct operational levels:

- **Chapter 1: The Single-Writer Kernel (Ground Truth).** We construct the minimal deterministic local kernel using SQLite WAL mode, establishing serializability, synchronous state decidability, and crash durability across process failures.
- **Chapter 2: The Anchor Protocol (Capability Delegation).** We design cryptographically verifiable capability tokens (Harbor Cards) and prove that delegated authority can only narrow at every hop.
- **Chapter 3: The Sealed Harbor (Confinement & Leakage).** We construct isolated execution harbors that bound information leakage between distrustful principals under strict egress quotas.
- **Chapter 4: The Legible Swarm (Human Supervision).** We derive the information-theoretic cost of supervisory attention, proving how digests must retain evidence paths without drowning the operator in raw telemetry.
- **Chapter 5: From Spawn to Person (Durable Agency).** We define persistent agent identity, append-only outcome ledgers, and reputation mechanisms that survive process restarts.
- **Chapter 6: The Harbor Economy (Markets & Clearing).** We establish bilateral clearing, underwriting reserves, and dynamic pricing for scarce host resources.
- **Chapter 7: The Bonded Commons (Game-Theoretic Governance).** We formalize claim signaling as an incentive-compatible repeated game, proving when cooperation is sustained under bonded escrow.
- **Chapter 8: The Federated Harbor (Cross-Machine Consensus).** We extend governance across distrustful host boundaries via witness logs and sheaf-theoretic consistency without central sovereignty.

References

- [1] G. Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, Cambridge, MA, 1986. 16
- [2] George A. Akerlof. The Market for “Lemons”: Quality Uncertainty and the Market Mechanism. *Quarterly Journal of Economics*, 84(3):488–500, 1970. 9
- [3] K. J. Arrow. *Social Choice and Individual Values*. John Wiley & Sons, New York, 1951. 26
- [4] J. L. Austin. *How to Do Things with Words*. Oxford University Press, 1962. 18
- [5] Lisanne Bainbridge. Ironies of Automation. *Automatica*, 19(6):775–779, 1983. 6
- [6] G. S. Becker. Crime and punishment: An economic approach. *Journal of Political Economy*, 76(2):169–217, 1968. 9, 10
- [7] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrabie, and Mark Lentczner. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014. <https://doi.org/10.14722/ndss.2014.23212> 4
- [8] R. H. Bordini, J. F. Hübner, and M. Wooldridge. *Programming Multi-Agent Systems in AgentSpeak using Jason*. John Wiley & Sons, 2007. 14
- [9] M. E. Bratman. *Intention, Plans, and Practical Reason*. Harvard University Press, Cambridge, MA, 1987. 12, 15

- [10] Mike Burrows. The Chubby Lock Service for Loosely-Coupled Distributed Systems. In *Symposium on Operating Systems Design and Implementation (OSDI)*, 2006. 3
- [11] T. Charrier, F. Ouchet, and F. Schwarzentruher. *Big Brother logic: Reasoning about agents equipped with surveillance cameras in the plane*. In *Proc. 13th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 421–428, 2014. 23, 25
- [12] E. H. Clarke. *Multipart pricing of public goods*. *Public Choice*, 11(1):17–33, 1971.
- [13] FIPA. *FIPA ACL Communicative Act Library Specification*. Technical Report SC00037J, Foundation for Intelligent Physical Agents, 2002. 18, 20
- [14] A. Gibbard. *Manipulation of voting schemes: A general result*. *Econometrica*, 41(4):587–601, 1973. 26
- [15] T. Groves. *Incentives in teams*. *Econometrica*, 41(4):617–631, 1973.
- [16] C. Hewitt, P. Bishop, and R. Steiger. *A universal modular ACTOR formalism for artificial intelligence*. In *Proc. 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 235–245, 1973. 16
- [17] C. A. R. Hoare. *Communicating sequential processes*. *Communications of the ACM*, 21(8):666–677, 1978. 16
- [18] Leslie Lamport. *Time, Clocks, and the Ordering of Events in a Distributed System*. *Communications of the ACM*, 21(7):558–565, 1978. 3, 4
- [19] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. *Lost in the Middle: How Language Models Use Long Contexts*. *TACL*, 12:157–173, 2024. arXiv:2307.03172. 31
- [20] R. B. Myerson and M. A. Satterthwaite. *Efficient mechanisms for bilateral trading*. *Journal of Economic Theory*, 29(2):265–281, 1983.
- [21] Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990. 9, 27
- [22] P. J. G. Ramadge and W. M. Wonham. *Supervisory control of a class of discrete event processes*. *SIAM Journal on Control and Optimization*, 25(1):206–230, 1987. 35
- [23] Anand S. Rao and Michael P. Georgeff. *Modeling Rational Agents within a BDI-Architecture*. In *International Conference on Principles of Knowledge Representation and Reasoning (KR)*, pages 473–484, 1991. 13, 15
- [24] A. S. Rao and M. P. Georgeff. *BDI agents: From theory to practice*. In *Proc. 1st International Conference on Multi-Agent Systems (ICMAS)*, pages 312–319, 1995. 13
- [25] Jerome H. Saltzer and Michael D. Schroeder. *The Protection of Information in Computer Systems*. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. 4
- [26] M. A. Satterthwaite. *Strategy-proofness and Arrow’s conditions: Existence and correspondence theorems for voting procedures and social welfare functions*. *Journal of Economic Theory*, 10(2):187–217, 1975. 26
- [27] J. R. Searle. *Speech Acts: An Essay in the Philosophy of Language*. Cambridge University Press, 1969. 18

- [28] N. Shinn, F. Cassin, B. Labash, A. Gopinath, P. Narasimhan, and S. Yao. *Reflexion: Language agents with verbal reinforcement learning*. In *Proc. NeurIPS*, 2023. 32
- [29] Y. Shoham and K. Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2009. 25
- [30] R. G. Smith. *The Contract Net Protocol: High-level communication and control in a distributed problem solver*. *IEEE Transactions on Computers*, C-29(12):1104–1113, 1980. 20, 22
- [31] W. Vickrey. *Counterspeculation, auctions, and competitive sealed tenders*. *The Journal of Finance*, 16(1):8–37, 1961.
- [32] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. *ReAct: Synergizing reasoning and acting in language models*. In *Proc. ICLR*, 2023. 32
- [33] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. *Tree of Thoughts: Deliberate problem solving with large language models*. In *Proc. NeurIPS*, 2023. 32
- [34] B. Chen, Z. Zhang, N. Cui, and H. Sun. *Hypertree Planning: Scalable Deliberate Search over Language Model Reasoning Spaces*. In *Proc. International Conference on Learning Representations (ICLR)*, 2025. 33
- [35] M. E. Conway. *How do Committees Invent?* *Datamation*, 14(4):28–31, 1968. 39
- [36] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. In *Proc. 36th Conference on Neural Information Processing Systems (NeurIPS)*, 2022. 28
- [37] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, et al. *MetaGPT: Meta Programming for Multi-Agent Collaborative Framework*. In *Proc. International Conference on Learning Representations (ICLR)*, 2024. 39
- [38] Y. Huang, J. Tan, K. Wang, and L. Chen. *Six Sigma Agent Swarms: Defect-Free Software Engineering via Statistical Process Control*. In *Proc. 48th International Conference on Software Engineering (ICSE)*, 2026. 39
- [39] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. In *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023.
- [40] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu, M. Yasunaga, Y. Zhang, D. Narayanan, Y. Wu, A. Kumar, et al. *Holistic Evaluation of Language Models*. *Transactions on Machine Learning Research (TMLR)*, 2023. 33
- [41] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, et al. *AgentBench: Evaluating LLMs as Agents*. In *Proc. International Conference on Learning Representations (ICLR)*, 2024. 33
- [42] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. *Training language models to follow instructions with human feedback*. In *Proc. 36th Conference on Neural Information Processing Systems (NeurIPS)*, 2022.

- [43] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. In *Proc. 37th Conference on Neural Information Processing Systems (NeurIPS)*, 2023. 30
- [44] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. *Attention Is All You Need*. In *Proc. 31st Conference on Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017. 28
- [45] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. arXiv:2305.16291, 2023. 32
- [46] Z. Wang, C. Gao, Z. Yuan, and Y. Shen. *TDAG: A Dynamic Topological Directed Acyclic Graph Framework for Multi-Agent Planning*. In *Proc. International Conference on Learning Representations (ICLR)*, 2025. 33
- [47] Y. Xie, K. Kawaguchi, Y. Zhao, X. Shen, M. Ding, and Q. Gao. *A Survey on Large Language Model Based Autonomous Agents*. *Frontiers of Computer Science*, 19(5):195301, 2025. 32
- [48] X. Zhou, R. Zhang, Y. Tian, and M. Zhang. *FormalJudge: Provably Sound Verification and Evaluation of Agentic Trajectories*. In *Proc. 40th AAAI Conference on Artificial Intelligence*, 2026. 33